

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA E
MATEMATICA APPLICATA



Corso di Laurea in Ingegneria Informatica (L-8)

**Sviluppo di un'applicazione di supporto alla guida:
progettazione e implementazione di un frontend per
dispositivi mobili**

Relatore:
**Ch.mo Prof.
Nicola Capuano**

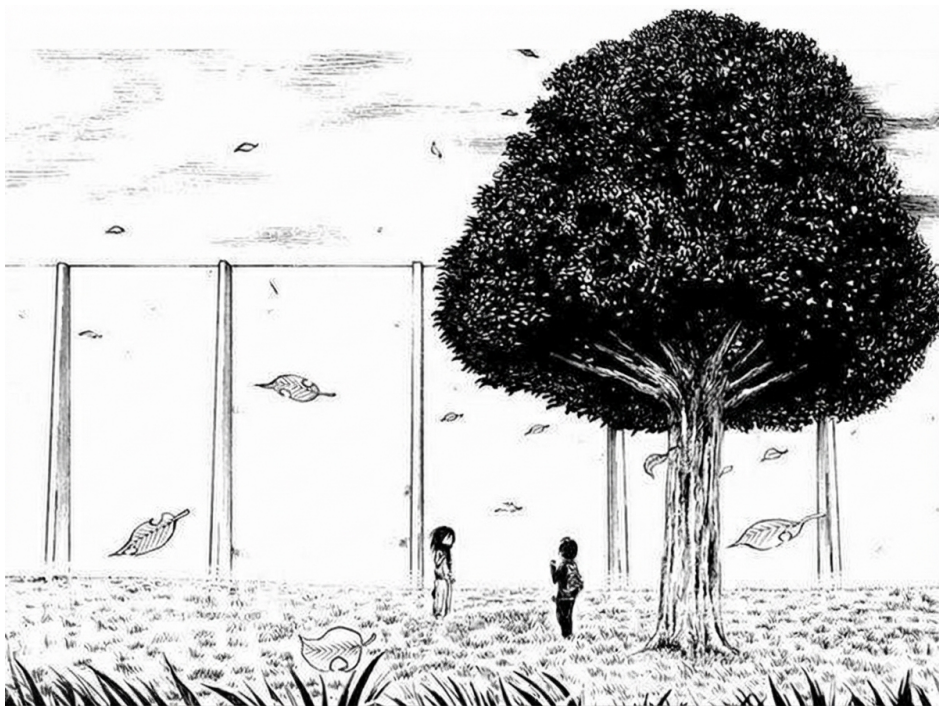
Candidato:
**Emanuele Tocci
Mat. 0612707488**

Tirocinio svolto presso:
MinervaS

ANNO ACCADEMICO 2024/2025

“A te, fra 2000 anni”

— *Hajime Isayama, L’attacco dei giganti*



Abstract	5
1 Obiettivi e tematiche di ricerca e sviluppo	5
1.1 Introduzione	5
1.1.1 Il problema della sicurezza e la distrazione	6
1.1.2 L'obsolescenza dei processi documentali	7
1.2 Obiettivo	7
1.3 Organizzazione dei contenuti	8
2 Tecnologie Abilitanti	9
2.1 Design e prototipazione	9
2.1.1 Figma	9
2.1.2 Miro	9
2.2 Linguaggi di programmazione e framework	10
2.2.1 Dart	10
2.2.2 Flutter	10
2.2.2.1 Architettura del framework	10
2.2.2.2 Ciclo di vita dei widget	11
2.2.3 Kotlin	12
2.2.4 Swift	12
2.3 IDE e tool di sviluppo	12
2.3.1 Android Studio	12
2.3.2 Xcode	12
2.3.3 Gradle KTS	12
2.4 Versioning e gestione del codice	13
2.4.1 Git	13
2.4.2 Github	13
2.5 Modelli e diagrammi tecnici	13
2.5.1 PlantUML	13
2.6 RESTful APIs ed SDKs	13
2.6.1 Google Maps APIs	13

2.6.2	API proprietarie sviluppate tramite <i>FastAPI</i>	14
2.7	Flutter packages	14
3	Progettazione ed implementazione del prototipo	15
3.1	Metodologia di sviluppo	15
3.2	I requisiti	16
3.2.1	Elicitazione e raccolta	16
3.2.2	Analisi dei requisiti	17
3.2.2.1	Categorizzazione e definizione delle priorità dei requisiti	17
3.2.2.2	Descrizione dei requisiti	18
3.3	I casi d'uso	20
3.3.1	I principali casi d'uso del sistema	21
3.3.2	Diagramma dei casi d'uso	27
3.4	Vista architetturale di massima	27
3.5	Progettazione di dettaglio ed Implementazione	29
3.5.1	Provider ed Observer	29
3.5.2	Modelli ed AppState	29
3.5.3	Servizi	30
3.5.4	Interazioni Principali	33
3.5.4.1	Flusso di onboarding e validazione dati	34
3.5.4.2	Avvio della navigazione e geocoding	35
3.5.4.3	Interazione vocale	35
3.6	Progettazione UI/UX: design for safety ed approccio alla sicurezza	36
3.6.1	La legge di <i>Fitts</i>	37
3.6.2	La " <i>Color Palette</i> " e la psicologia dei colori	39
4	Funzionamento del prototipo	40
4.1	Scenario applicativo	41
4.2	Dati	42
4.3	Verifica e validazione dei requisiti	43
4.4	Conclusioni e sviluppi futuri	44

Abstract

Nel contesto della mobilità professionale e della logistica, la *sicurezza stradale* e l'*efficienza operativa* rappresentano obiettivi primari spesso minacciati dalla *distrazione del conducente*. L'uso dello *smartphone* alla guida, divenuto ormai imprescindibile per la gestione dei flussi informativi, costituisce oggi una delle principali cause di incidenti, evidenziando la necessità di soluzioni tecnologiche che riducano il carico cognitivo e visivo sull'operatore.

L'*efficienza operativa* invece, è limitata dalla *gestione cartacea*, fonte di errori e ritardi. Manca uno strumento integrato che colmi il divario tra *logistica digitale* e attività in cabina, permettendo all'operatore di gestire navigazione e flussi informativi senza distogliere l'attenzione, garantendo così alti standard di *sicurezza* e *produttività*.

L'elaborato unisce *trasformazione digitale* e *sicurezza stradale*, sfruttando *Flutter* e l'*Intelligenza Artificiale* per portare l'assistenza alla guida su dispositivi consumer. Lo *smartphone* diviene così un supporto proattivo: l'integrazione di *OCR* (Optical Character Recognition) e modelli *Speech-to-Text* e *Text-to-Speech* abilita un'interazione *hands-free*. Tale approccio riduce il *carico cognitivo*, garantendo sicurezza e conformità agli standard.

Il contributo personale copre l'intero ciclo di sviluppo: dall'analisi dei requisiti alla progettazione UI mediante un approccio *safety-first*. Sviluppato in *Dart/Flutter*, il sistema integra scansione documentale, navigazione e comandi vocali. L'architettura adotta il pattern *Provider* per la gestione dello stato e un layer di servizi per le comunicazioni *RESTful*, garantendo *reattività*, *estensibilità* e *manutenibilità*.

Il *prototipo finale* gestisce l'intero ciclo operativo del trasporto su gomma. Integra una fase di *onboarding automatizzata* che, tramite *OCR*, estrae i dati dal *documento di trasporto* e dalla *carta di circolazione*, avviando la *navigazione turn-by-turn* con supporto vocale. I test confermano che l'*interazione bidirezionale* garantisce un accesso efficace alle informazioni, riducendo drasticamente le distrazioni e incrementando la sicurezza.

Obiettivi e tematiche di ricerca e sviluppo

1.1 Introduzione

Negli ultimi anni, gli smartphone si sono evoluti da semplici telefoni a veri e propri *hub personali*, divenendo strumenti essenziali per la produttività e la comunicazione quotidiana. Questa centralità tecnologica ha generato l'aspettativa di mantenere un livello costante di connettività in qualsiasi contesto, portando a una naturale estensione del loro impiego anche all'interno del settore *automotive*. È proprio in questo ambiente, tuttavia, che il ruolo dello smartphone assume una dimensione ambivalente: se da un lato la sua integrazione con il veicolo consente di offrire servizi innovativi e funzionalità avanzate, dall'altro l'interazione diretta con il dispositivo durante la guida, introduce un *fattore di rischio* significativo per la sicurezza stradale, rappresentato dalla distrazione del conducente.

Nel contesto del *trasporto merci su gomma* — anello fondamentale della catena logistica globale — questo fenomeno assume una dimensione ancora più critica. Mentre i centri di smistamento hanno raggiunto elevati livelli di digitalizzazione, l'operatività quotidiana del singolo autista rimane spesso ancorata a processi obsoleti. La cabina di guida si è trasformata, di fatto, in un *ufficio in movimento*, dove l'autista non è chiamato soltanto alla conduzione del veicolo, ma deve gestire simultaneamente la comunicazione con la sede e una complessa mole di *documentazione amministrativa*.

Attualmente, la convergenza tra questi strumenti e le necessità operative genera un conflitto critico tra *efficienza* e *sicurezza*:

- La *gestione cartacea* obbliga l'operatore a manipolare fisicamente i documenti per reperire informazioni cruciali, introducendo inefficienze e tempi morti.
- L'uso di smartphone “*general purpose*”, pur offrendo connettività, impone interfacce touch-screen complesse che richiedono un'attenzione visiva e manuale incompatibile con la guida sicura, introducendo un significativo fattore di rischio legato alla *distrazione del conducente*.

In questo scenario, la distrazione alla guida non è solo un fenomeno comportamentale, ma la conseguenza diretta di un *gap tecnologico*: la mancanza di strumenti che permettano di accedere alle informazioni di lavoro senza staccare le mani dal volante.

1.1.1 Il problema della sicurezza e la distrazione

La sicurezza stradale rappresenta una priorità assoluta nella mobilità moderna, dove la distrazione del conducente costituisce la causa principale degli incidenti. Numerose ricerche, come lo studio condotto da *Jeanne Breen*, intitolato *Car telephone use and road safety* [1] e commissionato dall'*Unione Europea*, hanno evidenziato come l'uso del telefono durante la guida possa determinare una riduzione considerevole dell'attenzione del conducente (1.1), portando a tempi di reazione più lenti fino al 50%, rispetto alla guida con un tasso alcolemico pari allo 0.00%.

Tra le varie forme di distrazione, la *distrazione cognitiva* è la più insidiosa e pericolosa: si manifesta nel momento in cui il cervello è impegnato in più attività mentali simultaneamente, come una semplice conversazione telefonica. Pur essendo spesso sottovalutata rispetto a quelle fisica, visiva o uditiva, essa comporta una riduzione significativa della capacità di concentrazione e di giudizio, rendendo il conducente meno reattivo e meno attento agli imprevisti della strada, mentre è coinvolto in attività mentali parallele alla guida.

Tabella 1.1: La tabella riporta le stime del rischio di incidente derivante da attività secondarie tra conducenti di autovetture, calcolate sulla base di dati provenienti da studi di osservazione del comportamento alla guida

Activity	Odds ratio (95% CI)	% driving time
Primarily cognitive activities		
Hand-held phoning (talking/listening)	1.27 (0.79–2.04)	2.7%
Hands-free phoning (talking/listening)	0.4 (0.10–1.63)	0.9%
Talking/singing alone	1.44 (0.99–2.08)	4.2%
Interacting with passengers	1.26 (0.98–1.62)	12.2%
Activities with in-vehicle devices		
Adjusting radio	1.57 (0.85–2.91)	1.3%
Controlling temperature/air conditioning*	2.3 (1.1–5.0)	0.56%
Interacting with vehicle device (other)*	4.6 (2.9–7.4)	0.83%
Mobile phone activities		
Purely holding a phone in the hand	2.05 (1.13–3.73)	1.1%
Reaching for phone*	4.8 (2.7–8.4)	0.58%
Dialling (hand-held)*	12.2 (5.6–26.4)	0.14%
Reading/writing text messages (texting)*	6.1 (4.5–8.2)	1.91%
Browsing (e.g. read email, check internet)*	2.7 (1.5–5.1)	0.73%
Other activities		
Reading/writing (also tablet)*	9.9 (3.6–26.9)	0.09%
Reaching for an object (no phone)*	9.1 (6.5–12.6)	1.08%
Prolonged looking at external object*	7.1 (4.8–10.4)	0.93%
Eating*	1.8 (1.1–2.9)	1.90%
Drinking (no alcohol)*	1.8 (1.0–3.3)	1.22%
Personal hygiene (e.g. make-up)*	1.4 (0.8–2.5)	1.69%
Child in rear seat*	0.5 (0.1–1.9)	0.80%

Fonte: European Road Safety Observatory, Road Safety Thematic Report: Driver Distraction, 2022. [2]

La *distrazione visiva*, invece, avviene quando l'autista distoglie lo sguardo dalla strada, ad esempio per interagire con il dispositivo mobile, aumentando così significativamente il rischio

di incidenti. Il tempo in cui gli occhi non sono concentrati sulla guida rappresenta un fattore critico che può compromettere la sicurezza in modo immediato.

1.1.2 L'obsolescenza dei processi documentali

La *sicurezza* non è l'unica sfida critica nel settore del trasporto merci. Un'altra problematica rilevante è rappresentata dalla *gestione obsoleta dei flussi informativi*: gli autisti sono spesso costretti a gestire manualmente un quantitativo considerevole di *documenti cartacei*. Questa dipendenza dal cartaceo non solo rallenta le operazioni di carico e scarico, introducendo inefficienze nella catena logistica, ma costringe l'operatore a distogliere l'attenzione dalla guida per consultare indirizzi o dettagli di consegna, creando un *circolo vizioso tra inefficienza operativa e rischio stradale*.

In questo scenario, la *digitalizzazione dei processi di bordo* diventa un fattore determinante. L'adozione di tecnologie per la *acquisizione e strutturazione intelligente dei dati*, unita a *interfacce vocali intelligenti*, promette di trasformare la cabina di guida in un ambiente di lavoro più sicuro ed efficiente, dove l'informazione è disponibile "*just-in-time*" senza richiedere interazioni fisiche.

1.2 Obiettivo

L'obiettivo principale di questa tesi è la progettazione e lo sviluppo del *frontend* ¹ di un'applicazione mobile cross-platform destinata agli autisti di mezzi pesanti. Il sistema è concepito per ottimizzare l'intera gestione del ciclo di trasporto, dalla fase preliminare di preparazione (onboarding documentale) alla navigazione assistita, fino al monitoraggio dello stato della consegna.

Per rispondere alle criticità evidenziate nelle sezioni precedenti, la soluzione proposta integra tecnologie avanzate di *Intelligenza Artificiale* mirate a due scopi principali:

- *Efficienza Operativa*: l'integrazione di moduli *OCR* ² permette l'estrazione automatica delle informazioni dai documenti fondamentali di viaggio (Documento di Trasporto - DDT e Carta di Circolazione). Questo automatismo facilita le operazioni preliminari, riducendo drasticamente i tempi di inserimento dati e gli errori manuali.
- *Sicurezza Attiva*: l'implementazione di un'interfaccia vocale completa, basata su moduli di sintesi (*TTS*) ³ e riconoscimento vocale (*STT*) ⁴, abilita un'interazione *hands-free*. Questo approccio, unito a una progettazione UI ispirata ai principi del *Design for Safety* (chiarezza visiva, elementi ad alto contrasto), minimizza la necessità di interazione manuale e visiva con il dispositivo.

¹ *Frontend*: componente di un sistema software responsabile dell'interfaccia utente e dell'interazione diretta con l'utente finale

² *OCR - Optical Character Recognition*: tecnologia che consente di estrarre automaticamente il testo da immagini o documenti scansionati, rendendolo modificabile e ricercabile

³ *TTS - Text-To-Speech*: tecnologia che consente di convertire automaticamente un testo scritto in voce sintetica "parlata"

⁴ *STT - Speech-To-Text*: tecnologia che permette di trascrivere una voce parlata in testo digitale tramite riconoscimento vocale

In sintesi, il prototipo realizzato mira a dimostrare come l'uso sinergico di queste tecnologie possa trasformare lo smartphone da potenziale fonte di distrazione a strumento di supporto proattivo, garantendo al contempo la sicurezza del conducente e l'efficienza del flusso logistico.

1.3 Organizzazione dei contenuti

Il presente lavoro è articolato come segue:

Il *Capitolo 2* descrive nel dettaglio le tecnologie abilitanti utilizzate per la progettazione e l'implementazione del prototipo, con particolare attenzione al framework **Flutter** e ai servizi esterni integrati

Il *Capitolo 3* si concentra sulla fase di progettazione ed implementazione, illustrando i requisiti, le scelte architetturali e la struttura dei principali componenti software del sistema.

Il *Capitolo 4* presenta il funzionamento complessivo del prototipo, definendo i dati gestiti, lo scenario applicativo di riferimento e il ruolo dei diversi moduli nell'esperienza d'uso dell'autista; il capitolo si conclude con una discussione sui possibili sviluppi futuri e sulle estensioni funzionali del sistema realizzato.

Questa sezione offre una panoramica strutturata delle principali tecnologie adottate nello sviluppo dell'applicazione, illustrando non solo le loro caratteristiche fondamentali, ma anche il ruolo che ciascuna ricopre nell'architettura complessiva del sistema. L'approccio seguito mira a evidenziare i motivi della scelta di ciascuna tecnologia in base ai requisiti di progetto, alla facilità d'integrazione e agli standard di settore.

2.1 Design e prototipazione

2.1.1 Figma



Figma [3] è una piattaforma di design grafico e *prototipazione* basata su cloud, utilizzata principalmente per la progettazione di *interfacce utente* - (UI) e la creazione di prototipi interattivi di siti web e applicazioni.

2.1.2 Miro



Miro [4] è una piattaforma di collaborazione visiva che, tra le sue funzionalità, offre strumenti avanzati per la progettazione di *wireframe* ¹ e *wireflow* ². Consente di creare rapidamente schemi e prototipi a bassa fedeltà dell'interfaccia utente, facilitando la pianificazione strutturale dell'applicazione e la condivisione delle idee progettuali all'interno del team.

¹ *Wireframe*: Rappresentazione visiva schematica, solitamente a bassa fedeltà, che definisce la struttura, il layout e la gerarchia dei contenuti di un'interfaccia utente, escludendo gli elementi di design grafico come colori e immagini.

² *Wireflow*: Artefatto di design che combina la struttura visiva dei *wireframe* con la logica dei diagrammi di flusso. Viene utilizzato per rappresentare le interazioni complesse e i percorsi di navigazione dell'utente, mostrando come le diverse schermate sono collegate tra loro in risposta alle azioni intraprese.

2.2 Linguaggi di programmazione e framework

2.2.1 Dart



Dart [5] è un *linguaggio di programmazione open-source* sviluppato da Google nel 2011. Progettato per rappresentare un'alternativa moderna, performante e più sicura a *JavaScript*, nello sviluppo di applicazioni web. Dart si distingue per la sua sintassi *orientata agli oggetti*, oltre che per il supporto sia alla compilazione *Ahead Of Time - AOT* ³ sia a quella *Just In Time - JIT* ⁴.

2.2.2 Flutter



Flutter [6] è un *framework open-source* creato da Google per lo sviluppo *cross-platform* di applicazioni mobili. Si tratta di un *SDK completo mobile-first* che consente di realizzare interfacce native ad alte prestazioni, utilizzando un'unica *codebase* per le piattaforme Android, iOS, Web e Desktop.

2.2.2.1 Architettura del framework

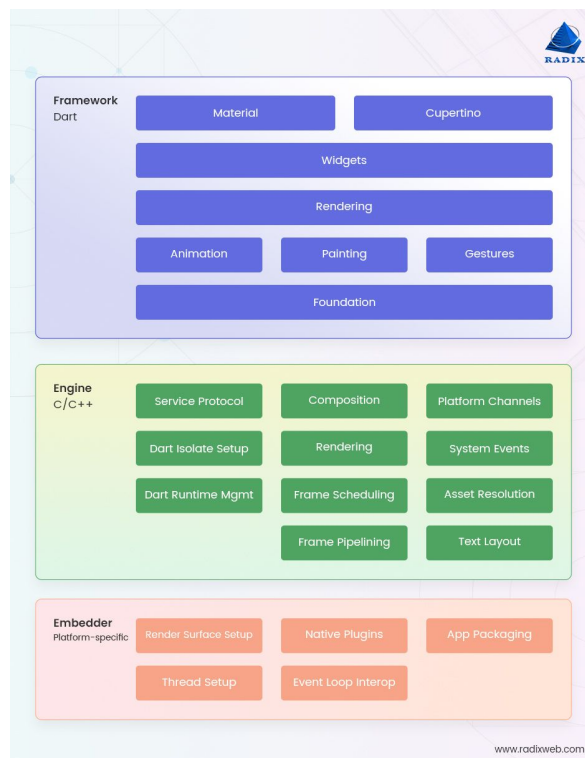


Figura 2.1: Schema a livelli dell'architettura di Flutter, composto dallo strato *framework* in Dart, dal *engine* in C/C++ e dal livello di *embedder* specifico per piattaforma. Fonte: Radix [7].

³AOT - *Ahead Of Time*: tecnica di compilazione che traduce il codice sorgente in codice macchina nativo prima dell'esecuzione del programma, migliorando generalmente le prestazioni e i tempi di avvio.

⁴JIT - *Just In Time*: tecnica di compilazione in cui la traduzione del codice in linguaggio macchina avviene dinamicamente durante l'esecuzione del programma (a *runtime*), anziché prima del suo avvio.

L'architettura di **Flutter** (mostrata in figura 2.1) è basata su un *widget tree* gerarchico, in cui ogni elemento dell'interfaccia (testi, pulsanti, layout, schermate) è rappresentato da un widget che può contenere a sua volta altri widget. L'applicazione viene eseguita su un motore grafico nativo ottimizzato (Flutter engine) che gestisce il rendering tramite una pipeline dedicata, consentendo di ottenere prestazioni paragonabili a quelle delle app native pur mantenendo un'unica *codebase cross-platform*.

Il codice dell'app è scritto in **Dart** e viene compilato in linguaggio macchina per le piattaforme mobili supportate, mentre il framework mette a disposizione un ricco set di widget predefiniti e di librerie per l'accesso ai servizi di sistema e alle API esterne.

2.2.2.2 Ciclo di vita dei widget

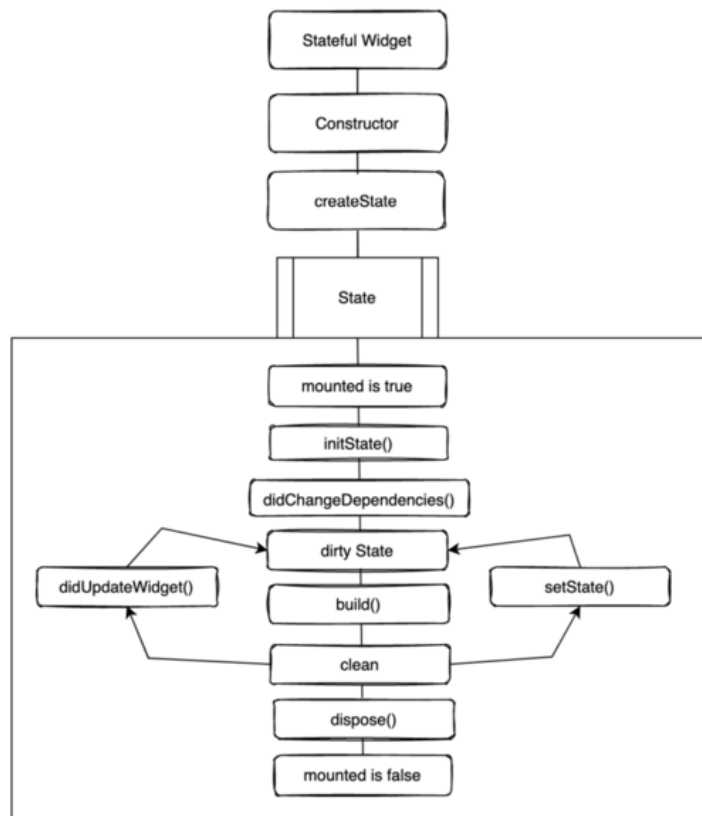


Figura 2.2: Schema del ciclo di vita di uno *StatefulWidget* in Flutter, che illustra le principali fasi di inizializzazione, aggiornamento e distruzione dello stato associato. Fonte: [8].

In Flutter esistono due principali tipologie di widget: *stateless*, privi di stato interno e usati per rappresentare porzioni di interfaccia immutabili, e *stateful*, associati a un oggetto **State** che mantiene le informazioni variabili nel tempo. Nel caso dei widget *stateless*, il ciclo di vita è essenzialmente limitato alla creazione dell'istanza e all'invocazione del metodo `build()`, che viene richiamato quando il framework deve (ri)costruire quella porzione dell'interfaccia.

Per i widget *stateful*, il ciclo di vita è più articolato e prevede fasi come `createState()`, `initState()`, `didChangeDependencies()`, le ricostruzioni successive attivate da `setState()` e, infine, `dispose()`, in cui vengono rilasciate le risorse associate al widget. Ogni chiamata a `setState()` marca lo stato come "dirty" e induce il framework a rieseguire il metodo `build()`,

integrando il widget nel processo di rendering che comprende costruzione dell'albero, layout e pittura dei pixel a schermo.

2.2.3 Kotlin



Kotlin [9] è un *linguaggio di programmazione* moderno, generalmente utilizzato per lo sviluppo nativo su *Android*. All'interno del progetto, Kotlin è stato impiegato esclusivamente per apportare alcune *modifiche di configurazione* specifiche nei file nativi, necessarie al corretto funzionamento dell'applicazione sui dispositivi Android.

2.2.4 Swift



Swift [10] è il linguaggio ufficiale di *Apple* per lo sviluppo nativo su *iOS*. Nel contesto del progetto, Swift è stato utilizzato soltanto per *gestire particolari personalizzazioni e configurazioni* nei file nativi di iOS, garantendo la piena compatibilità dell'app Flutter con i dispositivi Apple.

2.3 IDE e tool di sviluppo

2.3.1 Android Studio



Android Studio [11] è l'*ambiente di sviluppo* ufficiale per la piattaforma *Android*; durante la tesi è stato utilizzato per la creazione, il debug e il test dell'app Flutter su dispositivi Android ed emulatori, grazie ai suoi strumenti integrati e al supporto diretto per Flutter e Dart.

2.3.2 Xcode



Xcode [12] è l'*ambiente di sviluppo* fornito da *Apple* per la creazione di app destinate ai sistemi *iOS*; nel contesto del progetto, è stato impiegato per compilare, eseguire e testare l'app Flutter su dispositivi e simulatori iOS, gestendo anche la configurazione dei certificati e dei profili di sviluppo necessari alla distribuzione.

2.3.3 Gradle KTS



Gradle [13] è uno *strumento di automazione della build* utilizzato per gestire la compilazione e le dipendenze nella versione Android del progetto Flutter. *Gradle Kotlin DSL (KTS)* [14] è la sua configurazione scritta in *Kotlin* (file `.gradle.kts`). Nel corso dello sviluppo sono state effettuate modifiche manuali a questi file per garantire il corretto funzionamento di alcune funzionalità specifiche dell'applicazione.

2.4 Versioning e gestione del codice

2.4.1 Git



Git [15] è il *sistema di controllo versione* adottato durante lo sviluppo, fondamentale per tenere traccia delle modifiche apportate al codice, facilitare il lavoro collaborativo e mantenere l'integrità del progetto nel tempo.

2.4.2 Github



GitHub [16] è una piattaforma web utilizzata per lo *sviluppo collaborativo* di progetti software, basata sul sistema di controllo versione distribuito *Git*. Oltre a fungere da archivio online per i repository, facilita la collaborazione tra più sviluppatori, contribuendo all'organizzazione e all'efficienza nello sviluppo condiviso.

2.5 Modelli e diagrammi tecnici

2.5.1 PlantUML



PlantUML [17] è uno strumento *open-source* che consente di *generare diagrammi UML* ⁵ a partire da una semplice descrizione testuale. Utilizzato durante il progetto per la modellazione dell'architettura e dei flussi dell'applicazione, permette di creare rapidamente diagrammi di casi d'uso, sequenza, classi e attività, facilitando la comprensione e la comunicazione tra i membri del team di sviluppo.

2.6 RESTful APIs ed SDKs

2.6.1 Google Maps APIs



Le *Google Maps APIs* [18] sono un insieme di API ⁶ fornite da *Google* per l'interazione con i servizi offerti da *Google Maps*. Tramite gli SDK ⁷ e le API fornite, è stato possibile offrire un'esperienza di navigazione interattiva direttamente nell'applicazione, sfruttando sia i dati di geolocalizzazione che i servizi di *calcolo itinerario* forniti da Google. In particolare, sono stati utilizzati:

- *Directions API*
- *Geocoding API*
- *Maps Embed API*
- *Maps SDK for Android*
- *Maps SDK for iOS*
- *Navigation SDK*
- *Routes API*
- *Route Optimization API*

⁵UML - *Unified Modeling Language*: linguaggio di modellazione standardizzato general-purpose utilizzato nell'ingegneria del software per visualizzare, specificare, costruire e documentare gli artefatti di un sistema.

⁶API - *Application Programming Interface*: interfaccia che definisce le modalità di interazione tra componenti software, specificando funzioni, protocolli e regole che permettono la comunicazione tra programmi distinti.

⁷SDK - *Software Development Kit*: insieme di strumenti, librerie, documentazione e codice di esempio che facilitano lo sviluppo

2.6.2 API proprietarie sviluppate tramite *FastAPI*



Le API proprietarie sono state implementate utilizzando il framework *FastAPI* per *Python*, scelto per la sua efficienza nello sviluppo di servizi web *RESTful* ⁸ performanti e facilmente scalabili. Tali API consentono la gestione delle comunicazioni tra il frontend e i servizi di backend responsabili delle funzionalità di *OCR*, *TTS* ed *STT*.

2.7 Flutter packages

Nello sviluppo del prototipo sono stati adottati diversi *packages*, selezionati per semplificare l'implementazione delle funzionalità chiave dell'applicazione. Di seguito si riportano i più rilevanti, con una breve descrizione del ruolo svolto all'interno dell'applicazione.

- **google_fonts**: personalizzazione della tipografia dell'interfaccia tramite i font del catalogo Google Fonts.
- **path, path_provider**: gestione dei percorsi e accesso alle directory locali del dispositivo per il salvataggio temporaneo dei file.
- **flutter_doc_scanner**: scansione e ritaglio automatico dei documenti necessari alla fase di onboarding (DDT e carta di circolazione).
- **syncfusion_flutter_pdfviewer**: visualizzazione dei documenti PDF direttamente nell'applicazione, così da consentire la consultazione senza uscire dal flusso di lavoro.
- **provider**: gestione dello stato condiviso tra i widget, in linea con l'architettura a livelli adottata, per propagare in modo reattivo le variazioni dei dati nell'interfaccia.
- **http**: comunicazione con le API RESTful del backend per le funzionalità di OCR, TTS, STT e gestione dei dati di viaggio.
- **flutter_dotenv**: caricamento delle variabili d'ambiente (URL dei servizi, chiavi API, ecc.) da file esterni, mantenendo separate configurazione e codice sorgente.
- **location**: recupero della posizione GPS del veicolo, utilizzata per la navigazione e l'aggiornamento dinamico del percorso.
- **google_navigation_flutter**: integrazione del modulo di navigazione turn-by-turn di Google Maps direttamente all'interno dell'app.
- **audioplayers, flutter_sound**: riproduzione e registrazione di flussi audio per feedback vocali, avvisi sonori e interazione tramite comandi vocali.
- **permission_handler**: gestione centralizzata delle autorizzazioni di sistema (posizione, fotocamera, microfono, storage) necessarie al corretto funzionamento dell'app.
- **animated_text_kit**: animazione di testi e messaggi informativi, utilizzata per rendere più chiari e visibili alcuni feedback all'utente.

⁸*REST - Representational State Transfer*: aggettivo che definisce i servizi web conformi ai vincoli dell'architettura REST, caratterizzati da interazioni client-server stateless e dall'uso dei metodi HTTP standard per manipolare le risorse.

Progettazione ed implementazione del prototipo

Questo capitolo è dedicato all'intero processo di progettazione e sviluppo del *frontend* dell'applicazione mobile, realizzato seguendo i principi fondamentali dell'*Ingegneria del Software*. Vengono illustrate le scelte progettuali adottate, le metodologie seguite e le tecnologie impiegate per la realizzazione del prototipo. Particolare attenzione sarà posta all'analisi delle funzionalità chiave e all'architettura del sistema.

3.1 Metodologia di sviluppo

Data la complessità e le dimensioni del prototipo, la sua progettazione ha seguito un modello di processo tradizionale, prevedendo il controllo mediante feedback ad ogni fase. Il modello *Waterfall* (a cascata) (figura 3.1) [19] si basa su una successione ordinata di attività, in cui ciascuna fase riceve come input l'output della fase precedente. Questo implica che le diverse attività sono strettamente dipendenti l'una dall'altra in modo *sequenziale*: non è possibile procedere alla fase successiva senza che quella attuale sia stata completata e validata tramite apposita revisione.

La caratteristica principale del modello Waterfall risiede nella sua struttura fortemente ordinata ed intuitiva, che favorisce una chiara divisione del lavoro, semplifica la pianificazione e permette una gestione efficace delle risorse e dei tempi di progetto. Tuttavia, la rigidità del processo comporta alcune *criticità rilevanti*, maggiormente legate ai requisiti di sistema e alla gestione dei cambiamenti. Poiché ogni fase riceve come input l'output della fase precedente e necessita di essere completamente validata prima di procedere, eventuali modifiche ai requisiti richiedono di tornare indietro e rielaborare il lavoro già svolto nelle fasi antecedenti. Questo processo risulta estremamente oneroso sia dal punto di vista economico sia organizzativo, specialmente se i cambiamenti si verificano in uno stadio avanzato dello sviluppo.

Inoltre, tale modello, prevede la realizzazione di un'unica versione ("*release*") del sistema soltanto al termine dell'intero ciclo di sviluppo. Ciò implica che, fino alla conclusione di tutte le fasi progettuali e operative, non sono disponibili versioni intermedie del prodotto

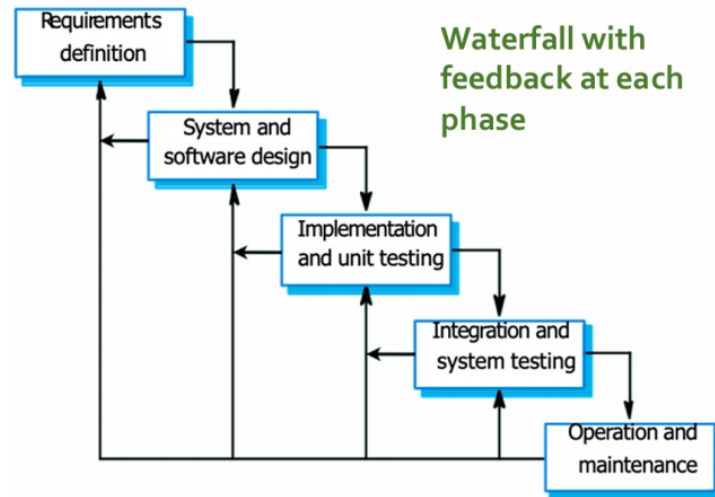


Figura 3.1: Modello a cascata con feedback su ogni fase

da utilizzare o valutare. Se il progetto viene interrotto prematuramente (ad esempio, per mancanza di fondi o per l'insorgere di altre problematiche) si rischia di non ottenere alcun risultato concreto né utilizzabile dalle attività svolte in precedenza.

L'introduzione dei *feedback* su ogni fase cerca di mitigare le debolezze del modello tradizionale, rispondendo all'esigenza di rendere il processo di sviluppo più reattivo e controllato. I feedback periodici vengono introdotti con l'obiettivo principale di intercettare tempestivamente eventuali errori o cambiamenti nelle esigenze degli utenti e degli *stakeholder* ¹. In questo modo, è possibile intervenire e correggere la direzione del progetto prima che i problemi si consolidino nelle fasi successive.

3.2 I requisiti

3.2.1 Elicitazione e raccolta

La fase di elicitazione e raccolta dei requisiti costituisce il momento iniziale e fondamentale all'interno del più ampio processo definito *Ingegneria dei requisiti* [19, cap.6]. L'ingegneria dei requisiti comprende una serie di attività strutturate e finalizzate a identificare, analizzare, documentare e gestire in modo sistematico tutte le funzionalità, le caratteristiche e i vincoli che il sistema dovrà rispettare nell'arco del suo ciclo di vita.

L'elicitazione mira a comprendere approfonditamente le necessità degli *stakeholder*, il dominio applicativo e le dinamiche operative, al fine di evitare fraintendimenti e assicurare una coerente definizione degli obiettivi progettuali.

L'approccio utilizzato per la raccolta di tali informazioni è stato quello delle *interviste* dirette.

¹ *Stakeholder*: termine che indica qualsiasi individuo, gruppo o organizzazione che ha un interesse (diretto o indiretto) in un progetto e che può influenzare le decisioni o essere influenzato dai risultati dello stesso (spesso tradotto come "*portatore di interesse*").

3.2.2 Analisi dei requisiti

La fase di *analisi e organizzazione* dei requisiti rappresenta un passaggio cruciale nel modello di processo a cascata, come già sottolineato nel paragrafo dedicato. In questa fase, i requisiti precedentemente raccolti vengono *esaminati* in modo approfondito e strutturato, con l'obiettivo di chiarire e dettagliare le funzionalità richieste, nonché di identificare eventuali ambiguità o incongruenze. L'analisi consente quindi di trasformare le esigenze grezze in *specifiche precise* e utilizzabili per le fasi successive di progettazione e sviluppo, garantendo una base solida e condivisa tra tutti gli stakeholder per l'intero progetto.

3.2.2.1 Categorizzazione e definizione delle priorità dei requisiti

I requisiti vengono *classificati e prioritizzati* secondo specifici criteri al fine di favorire una gestione più chiara e organizzata delle esigenze funzionali e non funzionali del sistema. Questa categorizzazione permette di identificare con precisione l'importanza e l'urgenza di ogni requisito, migliorando la pianificazione e guidando l'intera fase di sviluppo.

I requisiti vengono innanzitutto *categorizzati* in base alla loro natura, mediante un *prefisso* univoco rappresentante la relativa "*tipologia*", ed un numero del tipo $[X.Y]$ composto da due cifre ed un punto. Il primo valore $[X]$, rappresenta un'area di interesse e raggruppa pertanto requisiti comuni e/o afferenti alla stessa area tematica; il secondo valore $[Y]$ invece, specializza il requisito.

Successivamente, viene assegnata una *priorità* ai requisiti, valutando per ciascuno sia il *valore di business* che il *rischio tecnico*, al fine di determinare l'ordine ottimale di implementazione e indirizzare in modo efficace le attività di sviluppo sulle funzionalità più rilevanti o critiche.

La classificazione segue il seguente schema:

1. *Categorizzazione di dettaglio:*

- *IF*: Funzionalità individuali
- *DF*: Dati e formato dei dati
- *UI*: Interfaccia utente
- *IN*: Integrazione con sistemi esterni
- *NF*: Requisiti non funzionali

2. *Prioritizzazione* (BV ² / TR ³):

- *H*: Must-have / alto
- *M*: Should-have / medio
- *L*: Nice-to-have / basso

Nel presente lavoro si è scelto di applicare una categorizzazione di dettaglio solo ai requisiti funzionali, poiché rappresentano il nucleo principale delle funzionalità offerte dal prototipo e richiedono un maggiore livello di tracciabilità. I requisiti non funzionali (NF) sono invece

²*BV* - *Business Value*: rappresenta quanto un requisito è importante per il successo dell'applicazione, cioè quanto è utile o necessario per l'utente finale o per gli obiettivi aziendali.

³*TR* - *Technical Risk*: misura la complessità o rischiosità tecnologica di implementare un requisito

3. PROGETTAZIONE ED IMPLEMENTAZIONE DEL PROTOTIPO

in numero limitato e riguardano aspetti eterogenei ma trasversali, per cui una ulteriore suddivisione in sottocategorie sarebbe risultata poco significativa ai fini dell'analisi e avrebbe appesantito inutilmente la notazione adottata.

Tipo	Nome	BV	TR
IF-1	Procedura guidata di onboarding	H	M
IF-1.1	Scansione nativa dei documenti (DDT e carta di circolazione)	H	M
IF-1.2	Caricamento documenti pre-scansionati	H	M
IF-1.3	Cancellazione/Sostituzione di un documento caricato	H	L
IF-1.4	Identificazione dell'autista mediante dati personali	H	L
IF-2	Visualizzazione documenti caricati	H	L
IF-3	Supporto vocale continuo	M	H
IF-3.1	Implementazione di una parola di attivazione per l'assistente vocale	L	H
IF-4	Visualizzazione stato consegna	M	L
IF-5	Visualizzazione informazioni viaggio	M	L
IF-6	Sistema completo di navigazione	L	H
IF-6.1	Geocoding indirizzo	H	M
IF-6.2	Navigazione turn-by-turn	H	H
IF-6.3	Notifiche di navigazione e pericoli	M	M
IF-6.4	Aggiornamento dinamico del percorso	M	M
IF-7	Salvataggio temporaneo della sessione di guida corrente	H	L
IF-8	Cancellazione della sessione di guida corrente	H	L
UI-1	GUI semplice e minimale	H	L
UI-2	Layout responsive/adaptive	H	M
UI-3	Modalità notturna	M	L
DF-1	Interazione con il backend tramite scambio di dati in formato <i>JSON</i> ⁴	H	M
IN-1	Integrazione con servizi di backend containerizzati su server esterno	H	M
NF-1	Compatibilità multiplatforma (ios/Android)	M	M
NF-2	Dipendenza da connessione internet	H	L
NF-3	Leggibilità e usabilità in contesto di guida	H	M

3.2.2.2 Descrizione dei requisiti

IF-1: Procedura guidata di onboarding All'apertura dell'applicazione deve essere mostrata una *procedura di onboarding* che guida l'utente nell'utilizzo dell'applicazione. Tale procedura consente di:

1. Identificare l'autista mediante i propri dati personali
2. Specificare la rotta di percorrenza mediante il caricamento del *Documento Di Trasporto* - *DDT*
3. Specificare il mezzo utilizzato mediante il caricamento della *Carta Di Circolazione*

⁴JSON - JavaScript Object Notation: formato standard aperto per lo scambio di dati, che rappresenta le informazioni tramite una struttura leggibile composta da coppie chiave/valore e liste ordinate, ampiamente utilizzato per la comunicazione tra servizi web e applicazioni

3. PROGETTAZIONE ED IMPLEMENTAZIONE DEL PROTOTIPO

4. Verificare il completamento di tutti i dati inseriti attraverso una schermata riepilogativa prima di iniziare la sessione di guida

La procedura di onboarding è obbligatoria per poter accedere alle funzionalità principali dell'app e viene riproposta ad ogni nuovo utilizzo per garantire accuratezza e sicurezza del viaggio.

IF-2: Visualizzazione dei documenti caricati I documenti caricati devono essere visualizzabili attraverso una schermata apposita.

IF-3: Supporto vocale continuo L'applicazione deve fornire un supporto vocale continuativo tramite tecnologia TTS, comunicando tempestivamente all'autista tutte le informazioni rilevanti per il viaggio, così da minimizzare le distrazioni visive e favorire la sicurezza alla guida. Inoltre, deve essere possibile attivare l'assistente vocale in modo immediato tramite una parola chiave, garantendo un'interazione completamente hands-free.

IF-4: Visualizzazione stato consegna L'applicazione deve mostrare in modo chiaro e leggibile, lo stato attuale della consegna (es. *in corso*).

IF-5: Visualizzazione informazioni viaggio L'applicazione deve mostrare in modo chiaro e leggibile, le informazioni principali sul viaggio attuale, come l'indirizzo di destinazione, il mittente, il destinatario...

IF-6: Sistema completo di navigazione L'applicazione deve offrire un sistema completo di navigazione in modo tale che l'autista non debba interagire con applicazione esterne durante la guida. Tale sistema deve:

1. offrire un servizio di geocoding
2. offrire navigazione turno-per-turno, illustrando passo passo quale strada percorrere
3. offrire notifiche di navigazione vocale, soprattutto in caso di pericoli o traffico
4. aggiornare dinamicamente ed automaticamente il percorso in base alla posizione corrente del guidatore

IF-7: Salvataggio temporaneo della sessione corrente L'applicazione deve salvare in formato temporaneo tutte le informazioni riguardanti la sessione di guida corrente, come i dati relativi all'autista e i documenti caricati e di navigazione.

IF-8: Cancellazione della sessione di guida corrente L'applicazione deve garantire all'autista la possibilità di annullare o cancellare in qualsiasi momento la sessione di guida attiva, sia al termine della consegna sia in caso di immissione errata dei dati, così da consentire una gestione flessibile e sicura del proprio percorso.

UI-1: GUI semplice e minimale L'interfaccia grafica dell'applicazione deve presentare un design semplice e minimale, pensato per ridurre al minimo le distrazioni visive e garantire un'esperienza d'uso intuitiva e immediata, anche in situazioni operative complesse o in ambienti di lavoro dinamici.

UI-2: Layout responsive/adaptive L'interfaccia utente deve essere progettata secondo un layout responsive e adattivo, in grado di garantire una visualizzazione ottimale e un'esperienza d'uso coerente su dispositivi di diverse dimensioni e risoluzioni, che si tratti di smartphone, tablet o altri terminali mobili.

UI-3: Modalità notturna L'applicazione deve mettere a disposizione una modalità notturna che adatti automaticamente la palette cromatica e il contrasto dell'interfaccia alle condizioni di scarsa illuminazione, riducendo l'affaticamento visivo dell'utente e migliorando la sicurezza durante l'utilizzo in orario notturno.

DF-1: Interazione con il backend tramite scambio di dati in formato JSON Il flusso informativo tra applicazione e backend deve basarsi sullo scambio di dati strutturati in formato JSON, così da favorire interoperabilità, leggibilità e semplicità di integrazione fra i diversi componenti software della soluzione.

IN-1: Integrazione con servizi di backend containerizzati su server esterno L'applicazione deve poter comunicare con servizi di backend esposti tramite container su server remoto, utilizzando API REST standardizzate per garantire scalabilità, manutenibilità e isolamento dei diversi moduli funzionali del sistema

NF-1: Compatibilità multiplatforma La soluzione deve risultare completamente compatibile sia con dispositivi Android sia con dispositivi iOS, assicurando la stessa qualità funzionale e grafica su entrambe le piattaforme, grazie all'impiego di tecnologie di sviluppo cross-platform.

NF-2: Dipendenza da connessione internet L'applicazione richiede obbligatoriamente una connessione internet attiva per poter funzionare correttamente, in particolar modo per utilizzare le funzionalità offerte dalle API di Google.

NF-3: Leggibilità e usabilità in contesto di guida Il sistema deve garantire un'elevata "*glanceability*" (leggibilità a colpo d'occhio). Tutte le informazioni critiche visualizzate durante la navigazione (es. prossima svolta, velocità) devono essere comprensibili dall'autista con distorsioni dello sguardo dalla strada inferiori ai 2 secondi (standard di sicurezza *NHTSA*)⁵.

3.3 I casi d'uso

Prima di presentare nel dettaglio i casi d'uso [20, cap. 9] del sistema, è opportuno introdurre brevemente il concetto di *caso d'uso* nel contesto dell'ingegneria del software. Un caso d'uso descrive uno *scenario*, ovvero una sequenza di passi che caratterizzano una specifica interazione tra un utente (o, più in generale, un *attore*) e il sistema, con l'obiettivo di evidenziarne le funzionalità offerte e le condizioni richieste per il loro corretto svolgimento. L'interazione prende avvio proprio dal rispetto delle *pre-condizioni*, cioè delle condizioni iniziali che devono

⁵*NHTSA* - *National Highway Traffic Safety Administration*, le linee guida stabiliscono criteri specifici per limitare la distrazione visiva, raccomandando che l'interazione con i dispositivi elettronici di bordo non richieda al conducente di distogliere lo sguardo dalla strada per più di 2 secondi consecutivi per singola occhiata.

essere soddisfatte affinché il caso d'uso possa avvenire; allo stesso modo, il completamento del caso d'uso porta al soddisfacimento delle *post-condizioni*, che definiscono lo stato atteso del sistema al termine dello scenario.

Per una maggiore chiarezza espositiva, si è scelto di suddividere i casi d'uso utilizzando una notazione gerarchica analoga a quella adottata per i requisiti. In questa convenzione, $UC - X$ indica un caso d'uso principale, mentre $UC - X.Y$ identifica specifici sotto-casi d'uso *correlati*, facilitando così la tracciabilità e la comprensione della struttura funzionale del sistema. La suddivisione adottata riflette la segmentazione delle principali schermate funzionali presenti nel prototipo, garantendo così coerenza tra la struttura dei casi d'uso e l'organizzazione dell'interfaccia applicativa.

In particolare, verranno discussi i seguenti casi d'uso:

1. *UC-1: Onboarding e preparazione alla guida*
 - (a) *UC-1.1: Inserimento dei dati personali*
 - (b) *UC-1.2: Scansione/Caricamento documenti*
 - (c) *UC-1.3: Verifica e riepilogo dati*
2. *UC-2: Gestione della sessione di guida*
 - (a) *UC-2.1: Avvio della navigazione assistita*
 - (b) *UC-2.2: Monitoraggio stato e dettagli consegna*
 - (c) *UC-2.3: Gestione documentale*
 - (d) *UC-2.4: Conclusione della sessione di guida*

Si segnala, inoltre, che, per motivi di semplicità espositiva, sono state volutamente omesse le complesse interazioni tra il *frontend* e il *backend* relative a tutte le funzionalità di sintesi vocale (*TTS*) e riconoscimento vocale (*STT*) implementate e presenti in ciascuna schermata dell'applicazione.

3.3.1 I principali casi d'uso del sistema

UC-1: Onboarding e preparazione alla guida

- *Attori:* Autista
- *Descrizione:* Il macro-caso d'uso rappresenta l'intera procedura di accesso iniziale all'applicazione da parte dell'autista, comprendendo l'inserimento dei dati personali, il caricamento/scansione dei documenti e la verifica/approvazione degli stessi. L'obiettivo è garantire che l'utente e i documenti necessari siano correttamente registrati prima di consentire l'avvio di una sessione di guida.
- *Precondizioni:*
 - Nessuna sessione di guida è attiva
- *Postcondizioni:*

3. PROGETTAZIONE ED IMPLEMENTAZIONE DEL PROTOTIPO

- I dati personali e i documenti richiesti dell'autista sono correttamente inseriti e validati
- L'utente può accedere alla fase di guida assistita
- *Flusso di eventi normali:*
 1. L'autista avvia l'applicazione
 2. L'autista inserisce i dati personali (nome, cognome, ID)
 3. L'autista carica o scansiona i documenti richiesti
 4. Il sistema effettua la validazione e propone una schermata riepilogativa
 5. L'utente conferma i dati ed è pronto ad avviare la navigazione
- *Estensioni:*
 1. L'utente omette dati obbligatori: il sistema blocca l'avanzamento e segnala i campi mancanti
 2. La scansione dei documenti fallisce: il sistema invita a ripetere l'operazione

UC-1.1: Inserimento dei dati personali

- *Attori:* Autista
- *Precondizioni:*
 - L'applicazione è avviata e pronta a ricevere l'input dell'utente.
- *Postcondizioni:*
 - I dati personali sono memorizzati per la sessione corrente
 - Viene mostrata la schermata per il caricamento dei documenti
- *Flusso di eventi normali:*
 1. L'utente inserisce le proprie informazioni personali
 2. Il sistema verifica la validità dei dati
 3. Il sistema salva i dati nella sessione corrente
- *Estensioni:*
 1. Se i campi inseriti non sono validi (per esempio è stato inserito solo il nome o solo il cognome), il sistema mostra un errore ed evidenzia i campi da correggere

UC-1.2: Scansione/Caricamento documenti

- *Attori:* Autista
- *Precondizioni:*
 - I dati personali sono stati acquisiti
- *Postcondizioni:*
 - Le informazioni estratte dai documenti caricati vengono salvate nella sessione corrente e pertanto associate allo specifico autista
 - Viene mostrata la schermata di riepilogo
- *Flusso di eventi normali:*
 1. L'autista carica/scansiona ciascun documento richiesto
 2. Il sistema invia la scansione al backend per l'elaborazione *OCR*
 3. Il sistema ottiene una risposta dal backend e verifica la correttezza dei dati estratti
 4. Il sistema salva le informazioni estratte nella sessione corrente
- *Estensioni:*
 1. L'utente omette il caricamento di un documento obbligatorio: il sistema blocca l'avanzamento e segnala i campi mancanti
 2. La connessione con il backend è interrotta: il sistema mostra un avviso e blocca l'avanzamento
 3. L'utente carica un documento non valido: il sistema mostra un avviso e blocca l'avanzamento

UC-1.3: Verifica e riepilogo dati

- *Attori:* Autista
- *Precondizioni:*
 - Dati personali e documenti sono stati correttamente acquisiti
- *Postcondizioni:*
 - Tutte le informazioni sono confermate oppure modificate
 - Viene visualizzata la schermata di navigazione
- *Flusso di eventi normali:*
 1. Il sistema mostra le informazioni riepilogative riguardanti i propri dati personali e le informazioni estratte dai documenti inseriti
 2. L'utente conferma e prosegue
 3. Il sistema valida i dati inseriti
- *Estensioni:*

1. È presente un dato errato: l'utente corregge il dato sbagliato, conferma e prosegue

UC-2: Gestione della sessione di guida

- *Attori:* Autista
- *Descrizione:* Il macro-caso d'uso rappresenta l'intero processo relativo alla gestione operativa della sessione di guida da parte dell'autista. Comprende l'avvio della navigazione assistita, il monitoraggio in tempo reale dello stato della consegna e dei dati di viaggio, la consultazione e gestione dei documenti associati e, infine, la conclusione della sessione una volta completato il percorso o la consegna. L'obiettivo è garantire all'utente un'esperienza continua, sicura e integrata durante tutte le fasi del viaggio, dal momento dell'avvio fino al reset finale del sistema per nuove sessioni
- *Precondizioni:*
 - Onboarding completato con esito positivo
 - Tutti i dati e i documenti richiesti sono validi
- *Postcondizioni:*
 - La sessione di guida è terminata o azzerata: l'app è pronta per una nuova sessione
- *Flusso di eventi normali:*
 1. L'autista seleziona "*Avvia Navigazione*"
 2. Il sistema calcola il percorso: la navigazione ha inizio
 3. Il sistema mostra, durante il viaggio, lo stato della consegna e guida l'autista turno per turno mediante notifiche vocali. Inoltre offre la possibilità di consultare i documenti caricati in formato .pdf
 4. L'autista seleziona "*Termina Sessione*" e l'app si resetta
- *Estensioni:*
 1. Anomalia di sistema o navigazione: l'utente riceve notifica e istruzioni di ripristino

UC-2.1: Avvio della navigazione assistita

- *Attori:* Autista
- *Precondizioni:*
 - Onboarding completato
 - Ricezione del segnale satellitare GPS
- *Postcondizioni:*
 - Navigazione attiva; percorso e feedback vocali in funzione
- *Flusso di eventi normali:*
 1. L'utente conferma l'avvio della navigazione

2. Il sistema effettua geocoding ⁶ dell'indirizzo estratto dal *documento di trasporto*
 3. Il sistema calcola il percorso verso la meta
 4. Il sistema mostra il percorso calcolato ed avvia i feedback vocali
- *Estensioni:*
 1. Geocoding fallito: il sistema mostra un avviso ed invita l'utente a modificare manualmente l'indirizzo di destinazione

UC-2.2: Monitoraggio stato e dettagli consegna

- *Attori:* Autista
- *Precondizioni:*
 - Navigazione attiva
- *Postcondizioni:*
 - Stato consegna e progressi visualizzati in tempo reale
- *Flusso di eventi normali:*
 1. L'autista consulta la schermata di stato viaggio

UC-2.3: Gestione documentale

- *Attori:* Autista
- *Precondizioni:*
 - Navigazione attiva
- *Postcondizioni:*
 - Documenti consultabili in formato .pdf
- *Flusso di eventi normali:*
 1. L'utente alla schermata contenente i documenti caricati
 2. Il sistema mostra lo stato *OCR* e l'orario di scansione
 3. L'utente clicca su un documento
 4. Il sistema mostra il documento in formato .pdf
- *Estensioni:*
 1. Documento assente o corrotto: l'app mostra il messaggio di errore

⁶Geocoding: processo che consente di convertire un indirizzo o una descrizione di località in coordinate geografiche (latitudine e longitudine), rendendo possibile la localizzazione precisa di un punto sulla mappa.

UC-2.4: Conclusione della sessione di guida

- *Attori:* Autista
- *Precondizioni:*
 - Sessione di guida attiva
- *Postcondizioni:*
 - Stato resettato; sistema pronto per nuova sessione
 - Viene mostrata la schermata iniziale
- *Flusso di eventi normali:*
 1. L'autista seleziona "*Termina Sessione*"
 2. Il sistema cancella i dati della sessione corrente

3.3.2 Diagramma dei casi d'uso

In figura 3.2 è riportato il *diagramma dei casi d'uso* del sistema, che descrive a livello concettuale le funzionalità principali messe a disposizione dell'autista e le relative interazioni con il sistema DriveAssistant. I casi d'uso sono organizzati in due macro-insiemi: da un lato l'onboarding e la preparazione alla guida, dall'altro la gestione operativa della sessione di guida, in modo da riflettere le due fasi fondamentali di utilizzo del prototipo.



Figura 3.2: Diagramma dei casi d'uso del sistema DriveAssistant, con evidenza delle funzionalità di onboarding e di gestione della sessione di guida

3.4 Vista architetturale di massima

La progettazione architetturale [19, cap. 7.2.1] definisce la struttura complessiva del sistema software, individuando i principali componenti, le loro responsabilità e le modalità con cui interagiscono tra loro e con l'ambiente esterno. In questa sezione viene descritta l'architettura del prototipo *DriveAssistant*, con particolare attenzione al ruolo del frontend mobile e alle interazioni con i servizi di backend dedicati alle funzionalità di supporto alla guida degli autisti di mezzi pesanti.

3. PROGETTAZIONE ED IMPLEMENTAZIONE DEL PROTOTIPO

La figura 3.3 mostra la suddivisione del sistema in *componenti software*, le quali descrivono la struttura logica dell'implementazione senza entrare nei dettagli delle singole classi o schermate. In particolare, il *Mobile Client (app)* è organizzato in due livelli logici: un *Presentation Layer*, che astrae l'insieme delle schermate principali e di onboarding e un *Services Layer*, che astrae il concetto di servizi applicativi e di integrazione con l'esterno attraverso il componente *NetworkServices*.

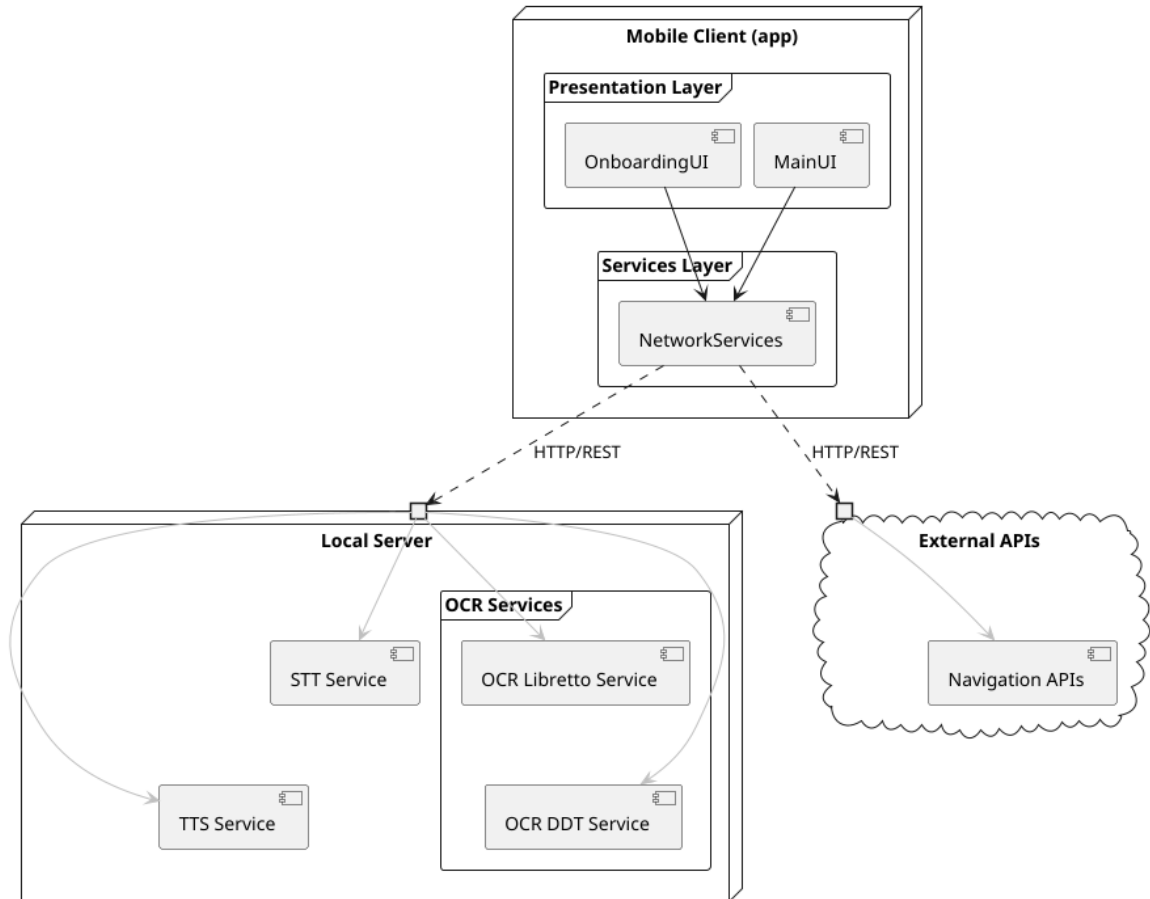


Figura 3.3: Vista a componenti del frontend *DriveAssistant*, con evidenza dei principali moduli dell'app mobile, dei servizi di backend AI ospitati sul server locale e delle API di navigazione esterne raggiunte tramite comunicazioni HTTP/REST.

Questa suddivisione riflette direttamente l'implementazione del frontend Flutter, in cui le varie schermate implementate, delegano ai vari servizi realizzati, tutte le operazioni che richiedono comunicazioni di rete. Il nodo *Local Server* rappresenta, in modo altrettanto astratto, il gruppo di servizi di backend, esposti tramite container docker (*STT*, *TTS*, *OCR*), mentre il blocco *APIs esterne* modella le API di navigazione di terze parti. Le connessioni HTTP/REST tra *NetworkServices* e questi due insiemi di servizi evidenziano come il frontend interagisca con l'infrastruttura server-side e con i provider esterni per realizzare le funzionalità di guida assistita.

3.5 Progettazione di dettaglio ed Implementazione

Dal punto di vista del frontend, l'applicazione è organizzata attorno alle principali funzionalità individuate in fase di analisi, senza introdurre pattern architetturali complessi come *MVC* o *MVVM*. La struttura del codice segue quindi una suddivisione per *tipologia* (approccio *package-by-layer*): le schermate di onboarding e di guida, i modelli di dominio e i servizi di integrazione sono raggruppati in package distinti, come mostrato nella figura 3.4.

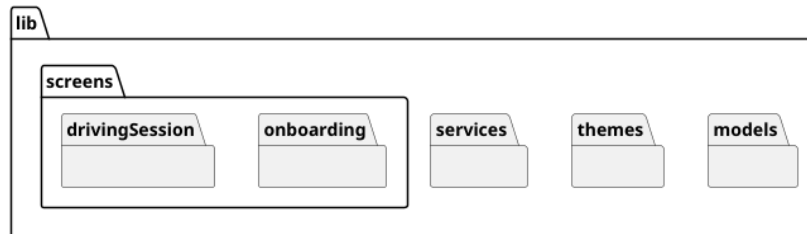


Figura 3.4: Organizzazione in package del progetto Flutter

Il package *screens* contiene le shell principali dell'interfaccia e le pagine dedicate alle diverse fasi di utilizzo (onboarding e sessione di guida), mentre i package *models* e *services* raccolgono rispettivamente i modelli applicativi e i servizi di accesso ai sistemi esterni. Il package *themes* completa l'organizzazione del progetto con gli stili grafici condivisi. Per i package *models* e *services*, che costituiscono il cuore della logica applicativa, nelle sezioni successive sono riportati specifici diagrammi UML delle classi, in modo da dettagliare la struttura interna e le principali responsabilità di ciascun componente.

3.5.1 Provider ed Observer

Per semplificare la gestione dello stato dell'applicazione, è stato adottato il pattern *Provider* [8, cap.16] che, nel contesto di **Flutter**, implementa sotto il cofano il pattern *Observer*. La classe **ChangeNotifier** rappresenta il soggetto osservato: mantiene lo stato della logica applicativa e notifica automaticamente tutti i listener (widget *Consumer*) ogni volta che avviene una modifica. Il widget **ChangeNotifierProvider** fornisce l'istanza del **ChangeNotifier** ai discendenti dell'albero dei widget, consentendo una condivisione semplice e trasparente dello stato tra le varie componenti dell'interfaccia grafica.

3.5.2 Modelli ed AppState

In questa sezione vengono presentati i principali modelli che compongono il dominio applicativo del prototipo e il relativo *AppState*. Le classi **DriverModel**, **DDTModel**, **LibrettoModel** e **NavigationStatusModel** rappresentano rispettivamente l'autista, i documenti di trasporto, il libretto di circolazione e lo stato della navigazione, fornendo una struttura tipata e coerente ai dati gestiti dall'applicazione. La classe **SessionModel** svolge il ruolo di *AppState* centralizzato: aggrega le istanze dei singoli modelli e, estendendo **ChangeNotifier**, notifica in modo reattivo i widget interessati ogni volta che una porzione dello stato viene aggiornata, mantenendo allineate tutte le schermate rispetto alla sessione di guida corrente.

Il diagramma UML in figura 3.5 sintetizza le relazioni tra queste classi e mette in evidenza il ruolo di **SessionModel** come punto di coordinamento dello stato condiviso. Come anticipato

3. PROGETTAZIONE ED IMPLEMENTAZIONE DEL PROTOTIPO

nel paragrafo precedente, tale stato applicativo è esposto all'albero dei widget e gestito a runtime tramite il pattern *Provider*, che consente di accedere in modo sicuro ed efficiente alle informazioni modellate dalle suddette classi.

```
1 class SessionModel extends ChangeNotifier {
2   DriverModel? _driver;
3   DDTModel? _ddt;
4   LibrettoModel? _libretto;
5   NavigationStatusModel? _navigationStatus;
6
7   SessionModel({
8     DriverModel? driver,
9     DDTModel? ddt,
10    LibrettoModel? libretto,
11    NavigationStatusModel? navigationStatus,
12  }) : _driver = driver,
13      _ddt = ddt,
14      _libretto = libretto,
15      _navigationStatus = navigationStatus;
16
17   void resetSession() {
18     driver = null;
19     ddt = null;
20     libretto = null;
21     navigationStatus = null;
22     notifyListeners();
23   }
24 }
```

Listato 1: Implementazione semplificata della classe `SessionModel` che gestisce lo stato condiviso della sessione di guida.

Nel listato 1 è riportato un estratto semplificato della classe `SessionModel`, che estende `ChangeNotifier` e centralizza le informazioni principali della sessione di guida (dati dell'autista, documenti e stato di navigazione). Il metodo `resetSession()` azzerava tali informazioni e invoca `notifyListeners()`, causando l'aggiornamento automatico dei widget che visualizzano i dati di sessione.

3.5.3 Servizi

Nel package `services` sono raccolte le classi che incapsulano le *interazioni* con i sistemi esterni, offrendo un'interfaccia ad alto livello al resto dell'applicazione. Ogni servizio è responsabile di un insieme omogeneo di operazioni ed espone metodi tipizzati che nascondono i dettagli delle chiamate HTTP/REST, della serializzazione in JSON e della gestione degli errori di rete.



Figura 3.5: Diagramma UML delle classi del package *models*

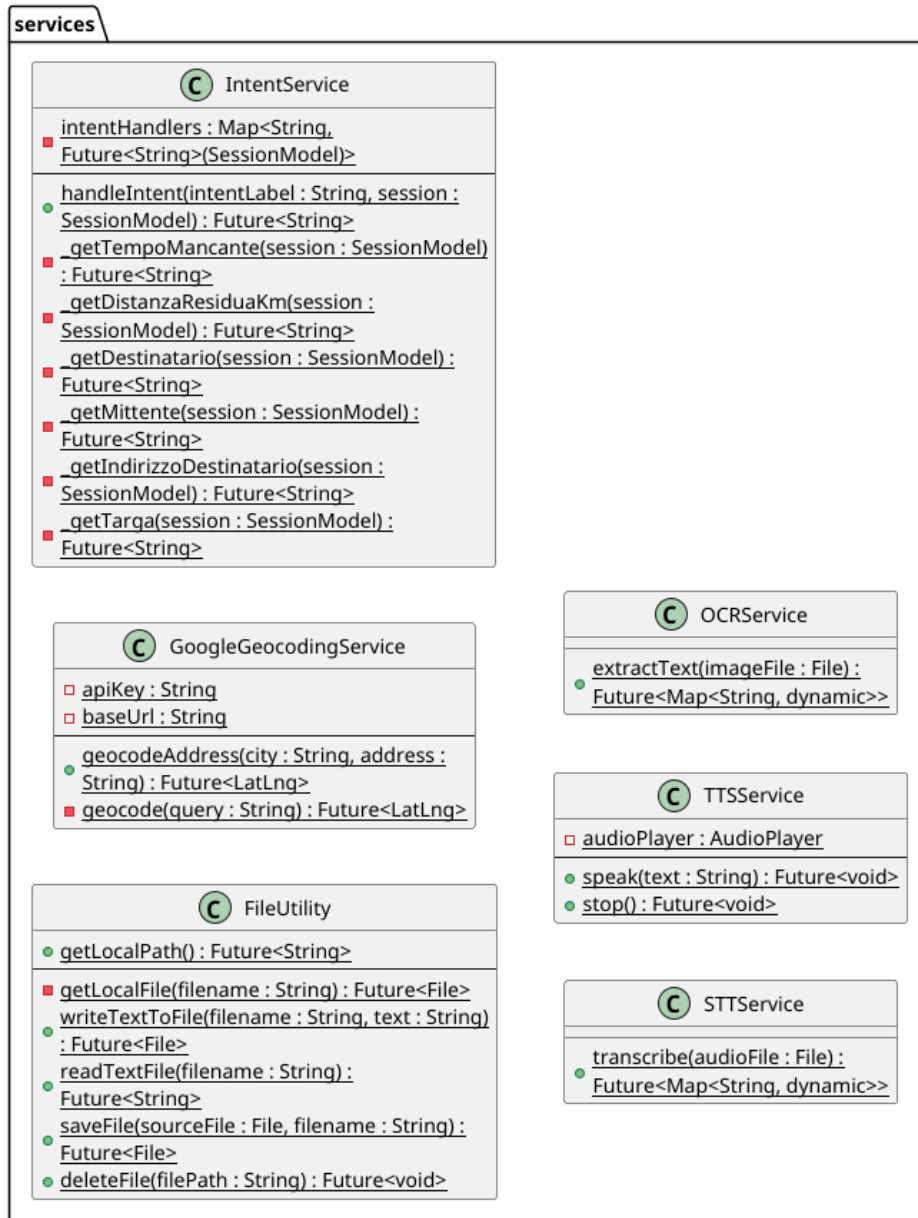


Figura 3.6: Diagramma UML delle classi del package *services*

Le classi **STTService** e **TTSService** gestiscono rispettivamente l'invio dei flussi audio al servizio di riconoscimento vocale (STT) e la richiesta di sintesi vocale (TTS) verso il backend AI in esecuzione sul server locale. Entrambi i servizi ricevono in ingresso dati già strutturati dai modelli applicativi e restituiscono risultati pronti per essere integrati nell'interfaccia (testo trascritto o messaggi vocali), consentendo alle schermate di delegare completamente la logica di comunicazione con i container remoti.

La classe **GoogleGeocodingService** si occupa invece di interfacciarsi con le Google Maps APIs per le operazioni di geocodifica e di costruzione del percorso. Le risposte dell'SDK vengono tradotte negli oggetti del modello di dominio, in particolare aggiornando il **NavigationStatusModel** e le strutture che rappresentano i segmenti di percorso, così che la logica di navigazione rimanga indipendente dal formato proprietario delle API esterne.

Un ulteriore servizio, dedicato all'elaborazione OCR, incapsula le chiamate HTTP/REST

verso i microservizi che estraggono automaticamente i dati dai documenti di trasporto e dal libretto di circolazione. Questo *OCR service* riceve in ingresso le immagini acquisite dall'app, inoltra le richieste al backend AI e traduce le risposte nei corrispondenti modelli di dominio (*DDTModel* e *LibrettoModel*), rendendo trasparente al frontend il formato dei payload e la logica di elaborazione.

Infine, il servizio *IntentService* centralizza la gestione degli *intent* dell'assistente vocale, instradando i comandi riconosciuti verso le azioni appropriate dell'applicazione (ad esempio avvio/arresto della navigazione o richiesta di lettura delle informazioni di viaggio), mentre la classe di utilità *FileUtility* fornisce metodi statici per salvare, leggere ed eliminare file nella directory locale dell'applicazione, consentendo l'accesso al file system e supportando la persistenza temporanea di documenti e dati di servizio.

3.5.4 Interazioni Principali

Per completare la descrizione dell'architettura software, è importante analizzare il *comportamento dinamico* del sistema nel tempo. Mentre il diagramma delle classi mostra la struttura statica, i *Diagrammi di Sequenza* rappresentano come i diversi componenti si scambiano messaggi e collaborano per soddisfare le funzionalità richieste. Questi diagrammi aiutano a comprendere il flusso operativo, illustrando nel dettaglio la sequenza temporale delle interazioni tra interfaccia utente, logica dell'applicazione e servizi esterni.

Al fine di facilitare la lettura dei flussi sequenziali proposti, si definiscono di seguito i ruoli e le responsabilità dei *partecipanti* coinvolti nelle interazioni.

1. *Autista*: rappresenta l'autista che interagisce con il sistema. Le sue azioni (tap, input vocali) costituiscono i *trigger* che avviano i flussi operativi;
2. *Widget*: rappresenta il *Presentation Layer* dell'applicazione (schermate Flutter, pulsanti, dialog). È responsabile della cattura degli input e dell'aggiornamento visivo dell'interfaccia in risposta ai cambiamenti di stato.
3. *ServiceManager*: questo componente astrae la logica di business e l'orchestrazione delle chiamate, coordinando il lavoro tra l'interfaccia utente, i servizi di rete e i modelli di dati. A livello implementativo coincide con le classi presenti nel package *services*.
4. *Backend*: Identifica genericamente l'infrastruttura server-side esterna all'applicazione mobile. Include sia i microservizi proprietari containerizzati (OCR, STT, TTS) che le API di terze parti (Google Maps Platform).
5. *SessionModel*: Rappresenta lo *stato applicativo locale* (implementato tramite il pattern *Provider*). Mantiene in memoria i dati persistenti della sessione corrente e garantisce la coerenza delle informazioni tra le diverse schermate.

3.5.4.1 Flusso di onboarding e validazione dati

La prima interazione critica riguarda l'*acquisizione dei dati di viaggio* (figura 3.7). Il diagramma mostra la gestione dei dati personali e il caricamento dei documenti: il sistema entra in un ciclo di validazione (loop) che termina solo quando i documenti (DDT e Carta di Circolazione) vengono correttamente analizzati dall'OCR. In caso di incongruenze, il flusso prevede un blocco di correzione manuale (evidenziato in rosso) che garantisce l'integrità dei dati prima di finalizzare la sessione.

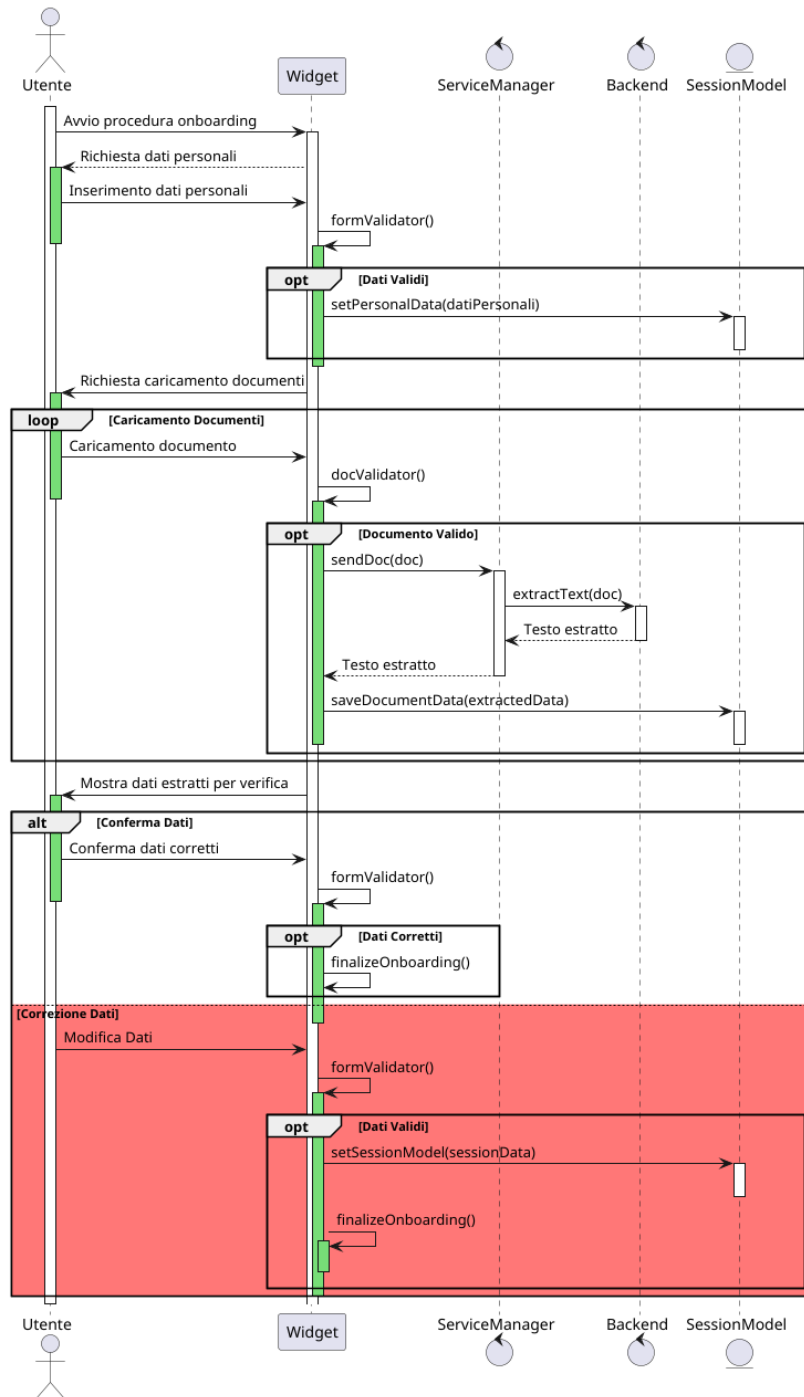


Figura 3.7: Diagramma di sequenza del processo di onboarding dell'utente, che illustra le principali interazioni tra interfaccia, servizi di estrazione dati e gestione della sessione.

3.5.4.2 Avvio della navigazione e geocoding

La Figura 3.8 descrive l'*inizializzazione della guida assistita*. Questo flusso trasforma l'indirizzo estratto dal documento cartaceo in una *rotta GPS*. Il diagramma evidenzia la robustezza del sistema: se il servizio di Geocoding non riesce a risolvere l'indirizzo (flusso alternativo in rosso), l'applicazione non si blocca ma richiede all'utente un inserimento manuale, aggiornando il **SessionModel** e ripetendo il tentativo di calcolo della rotta fino al successo.

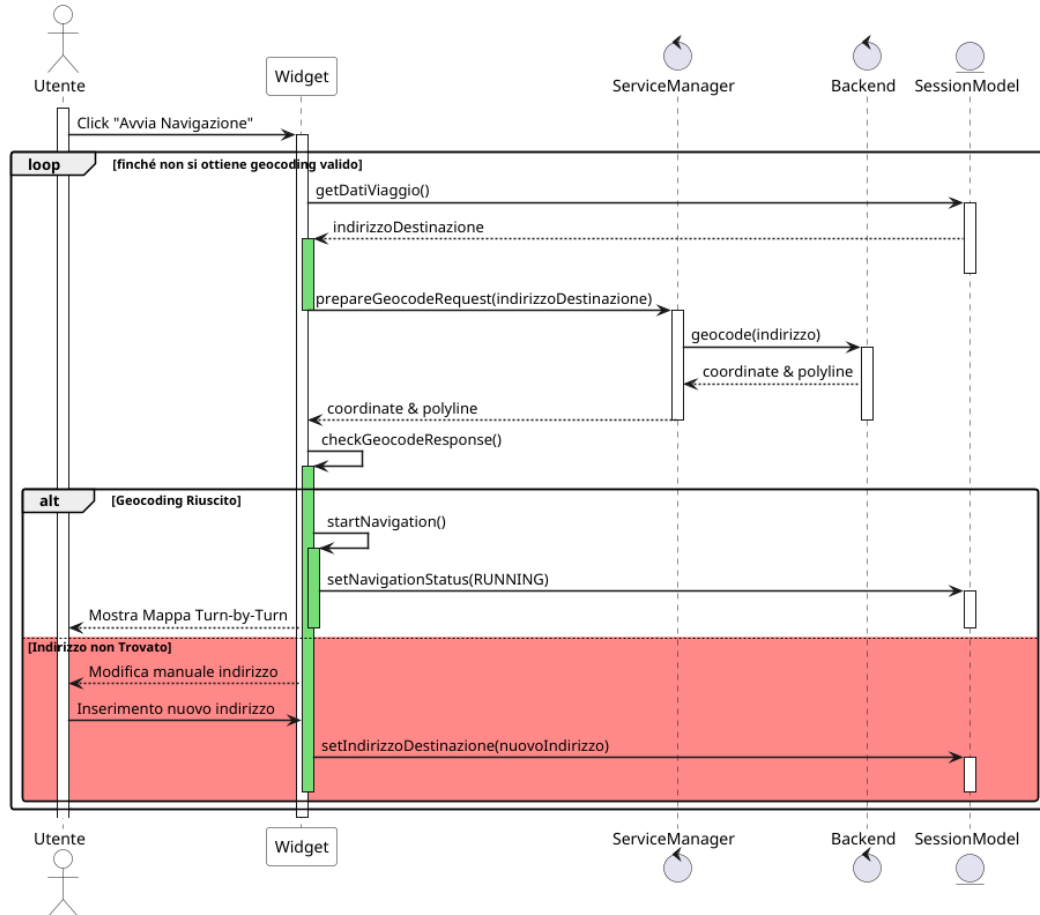


Figura 3.8: Diagramma di sequenza del processo di geocoding e avvio della navigazione assistita, che mostra le interazioni tra utente, interfaccia applicativa e servizi di calcolo dell'itinerario.

3.5.4.3 Interazione vocale

Infine, la figura 3.9 illustra il funzionamento dell'*assistente vocale*, elemento chiave per la sicurezza alla guida. Il flusso mostra come il comando audio venga inviato in streaming al backend per la trascrizione e l'estrazione dell'intento (*Intent Classification*). Particolare rilevanza assume l'*interazione* con il SessionModel: a seguito di una domanda correttamente classificata (con label diversa da *unknown*), l'assistente recupera automaticamente i dati di viaggio contestuali dalla sessione locale, per poi generare una risposta vocale personalizzata (TTS).

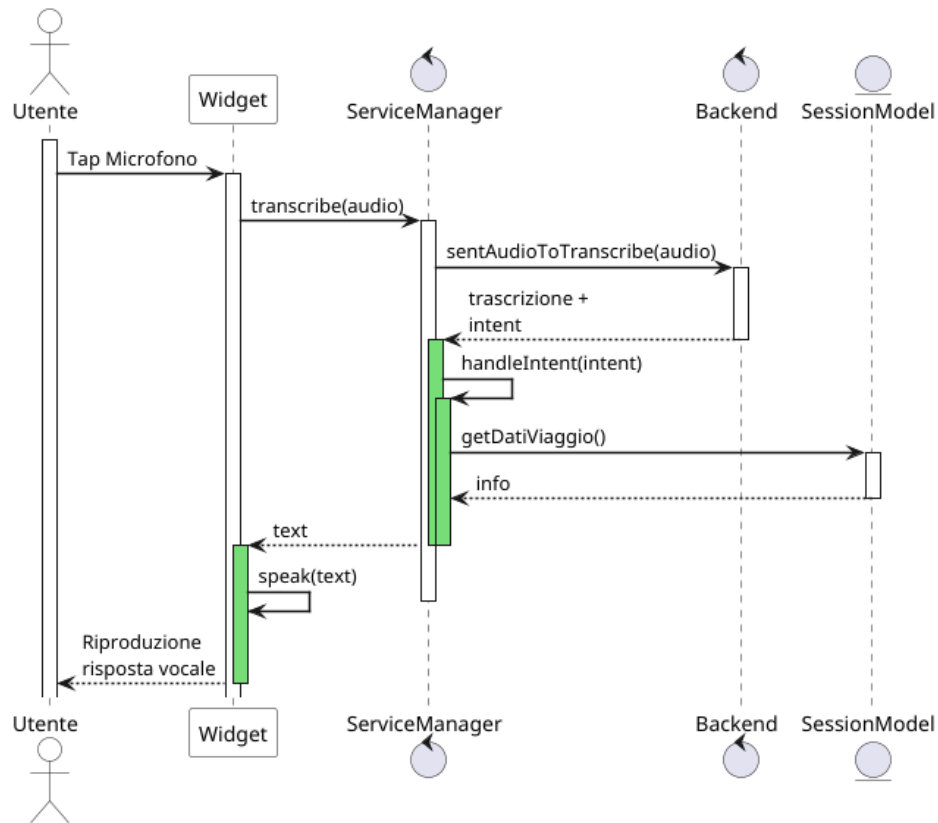


Figura 3.9: Diagramma di sequenza del flusso di interazione vocale, che rappresenta la gestione dei comandi vocali e della sintesi TTS tra utente, interfaccia e servizi dedicati al riconoscimento ed emissione audio.

3.6 Progettazione UI/UX: design for safety ed approccio alla sicurezza

La progettazione dell'interfaccia utente (UI)⁷ e dell'esperienza utente (UX)⁸ è stata affrontata con l'obiettivo di semplificare al minimo le interazioni, focalizzandosi sulle esigenze specifiche dell'utente durante la guida.

In particolare, l'interfaccia utente, è stata concepita per cercare di ridurre le distrazioni visive, privilegiando le interazioni vocali fornite mediante l'*Intelligenza Artificiale* e pulsanti grandi e ben visibili, per un utilizzo rapido ed intuitivo.

Dal punto di vista della *UX*, l'esperienza utente è stata studiata per rendere l'interazione fluida e coerente, guidando l'utente attraverso flussi logici prestabiliti, progettati per essere semplici ed intuitivi. Questo approccio ha garantito la facilità di accesso alle principali funzionalità dell'applicazione, attraverso un percorso di utilizzo lineare, che possa essere seguito anche in condizioni di attenzione limitata.

⁷ *UI - User Experience Design*: fase della progettazione che mira ad ottimizzare l'esperienza d'uso del prototipo, al fine di renderlo intuitivo e facilmente accessibile a tutti

⁸ *UX - User Interface Design*: fase della progettazione riguardante principalmente l'aspetto visivo del prodotto, come il posizionamento degli elementi grafici, i colori da utilizzare, la tipografia...

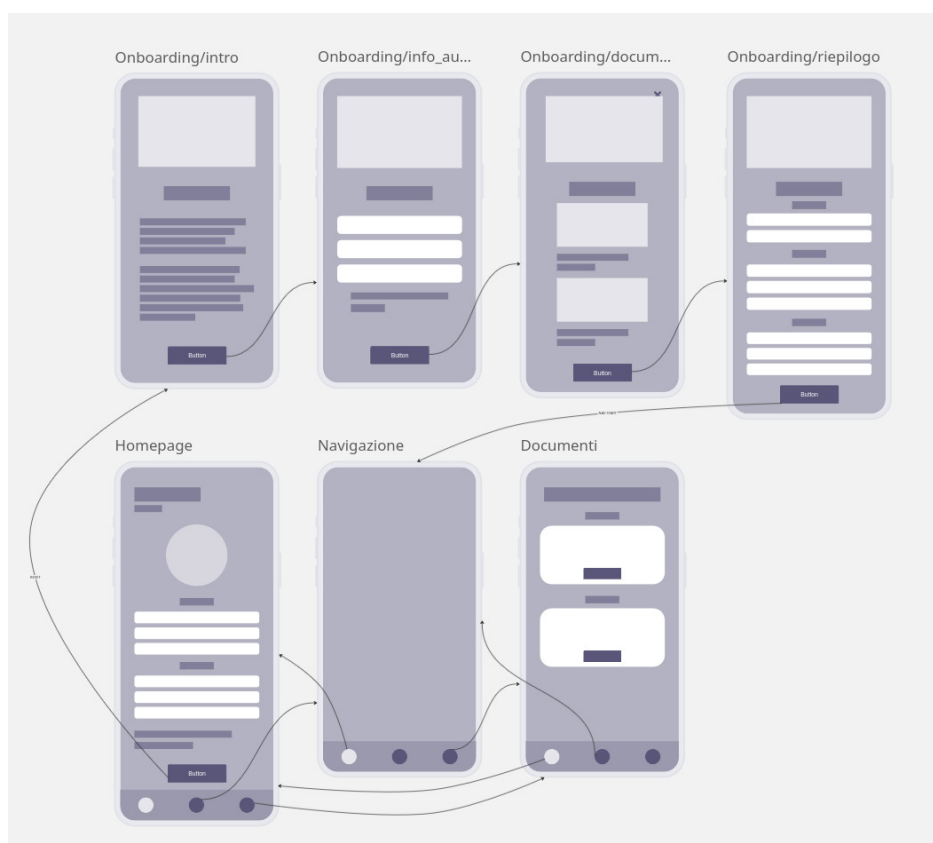


Figura 3.10: Wireflow a bassa fedeltà (*lo-fi*) che rappresenta il flusso di navigazione tra le principali schermate del prototipo.

L'interfaccia dell'applicazione si articola in due macro-aree funzionali: la fase di *onboarding* (fig.3.11), caratterizzata da un percorso guidato e obbligatorio, e le schermate di *navigazione* (fig.3.12), accessibili all'utente soltanto previo completamento di questa procedura propedeutica.

3.6.1 La legge di *Fitts*

Il design dell'applicazione è stato progettato per soddisfare i *principi di ergonomia dell'interazione*, in particolare la *legge di Fitts*, secondo cui il tempo necessario per selezionare un elemento dell'interfaccia dipende dalle sue dimensioni e dalla distanza percorsa dal dito. Per questo motivo, tutti i pulsanti e le aree cliccabili nelle schermate principali sono stati progettati con dimensioni generose e ben spaziati tra loro, così da facilitare l'interazione rapida e precisa anche in condizioni di movimento o vibrazione, tipiche del contesto di guida. L'utilizzo di *colori contrastanti* tra pulsanti e sfondo contribuisce ulteriormente a renderli immediatamente *riconoscibili* e accessibili.

3. PROGETTAZIONE ED IMPLEMENTAZIONE DEL PROTOTIPO

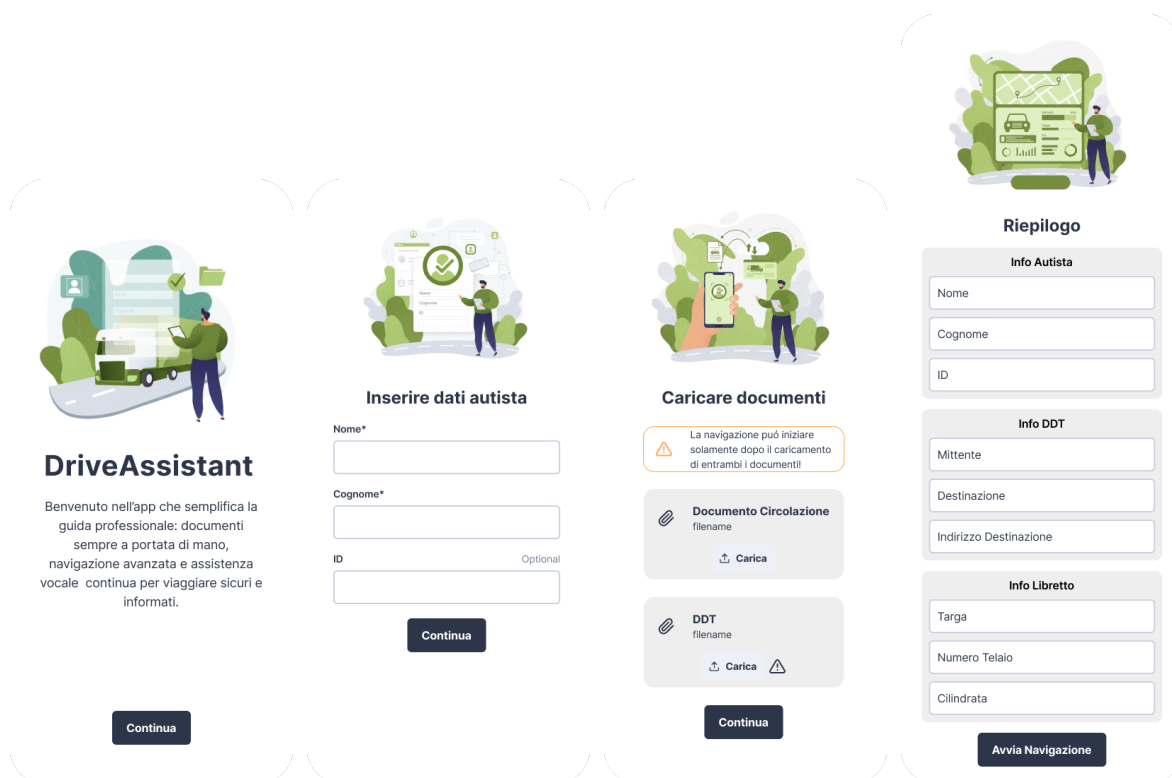


Figura 3.11: Mockup ad alta fedeltà (*hi-fi*) del flusso di *Onboarding* del prototipo: schermata introduttiva, inserimento dei dati dell'autista, caricamento dei documenti e riepilogo finale delle informazioni.

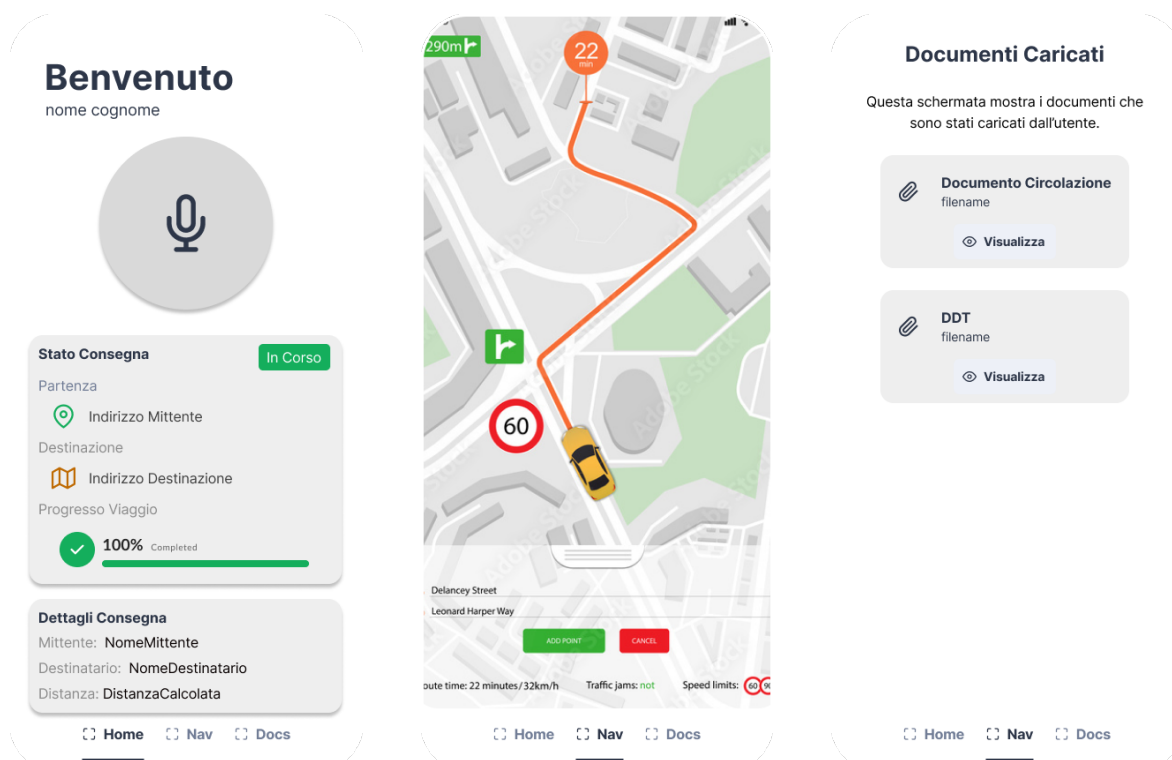


Figura 3.12: Mockup ad alta fedeltà (*hi-fi*) delle schermate di *navigazione* del prototipo: homepage, interfaccia di navigazione e sezione dedicata alla gestione dei documenti.

3.6.2 La "Color Palette" e la psicologia dei colori

La palette colori dell'applicazione è strutturata principalmente sulle tonalità del *verde* e del *turchese*, una scelta progettuale motivata sia da ragioni psicologiche (vedi fig. 3.13) sia funzionali. Il verde è universalmente percepito come un *colore rassicurante*, associato a sensazioni di *sicurezza*, stabilità e permesso ("*via libera*" nel codice della strada). Questa scelta favorisce un ambiente digitale rilassante, riducendo lo stress visivo e aumentando la percezione di affidabilità dell'interfaccia, qualità essenziale per utenti che devono mantenere alta la concentrazione durante l'utilizzo.

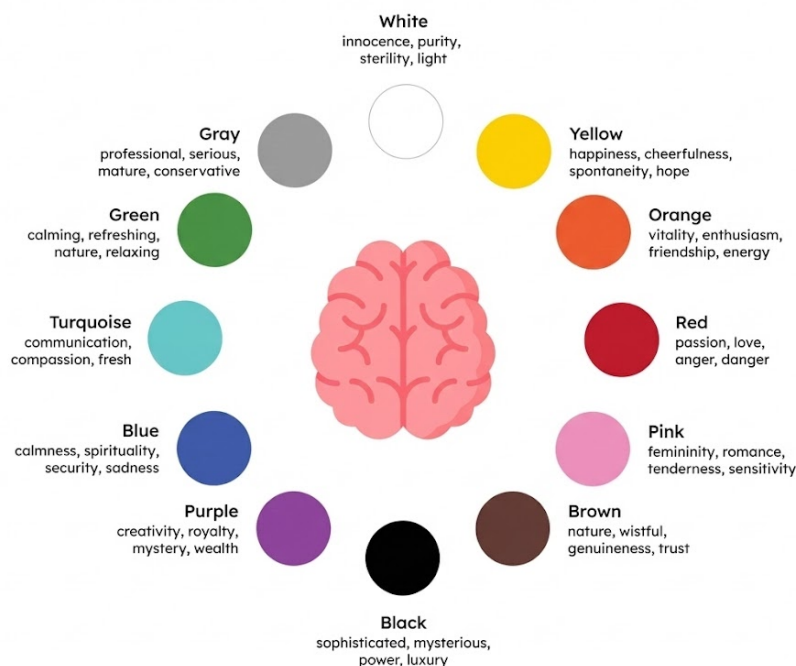


Figura 3.13: Mappa delle associazioni emozionali nella psicologia del colore [21]. Il diagramma mostra la correlazione tra spettro cromatico ed elaborazione cognitiva. Ogni nodo circolare rappresenta un colore primario o secondario, accompagnato dai descrittori psicologici chiave che definiscono l'esperienza percepita dall'utente.

Inoltre, l'applicazione supporta pienamente la modalità scura ("*dark mode*"), che consente di adattare automaticamente i colori di sfondo e degli elementi testuali agli ambienti a bassa luminosità. Questo non solo contribuisce a diminuire l'affaticamento degli occhi nelle ore serali o notturne, ma migliora anche la *leggibilità* dei contenuti e la visibilità delle notifiche più critiche.

Funzionamento del prototipo

Il presente capitolo descrive il *funzionamento complessivo* del prototipo sviluppato, mettendo in relazione i requisiti individuati, le scelte architetturali e le funzionalità esposte all'utente finale. L'obiettivo è mostrare come i moduli di onboarding documentale, supporto vocale e navigazione cooperino per fornire all'autista un'esperienza d'uso continua e il più possibile priva di distrazioni durante le fasi di preparazione e svolgimento del viaggio.

Nelle sezioni successive vengono innanzitutto presentati i dati gestiti dal sistema, quindi viene illustrato lo scenario applicativo di riferimento che segue il ciclo tipico di una consegna, dalla scansione dei documenti all'arrivo a destinazione. Il capitolo si conclude con una sintesi dei risultati ottenuti e con una panoramica dei possibili sviluppi futuri del prototipo.

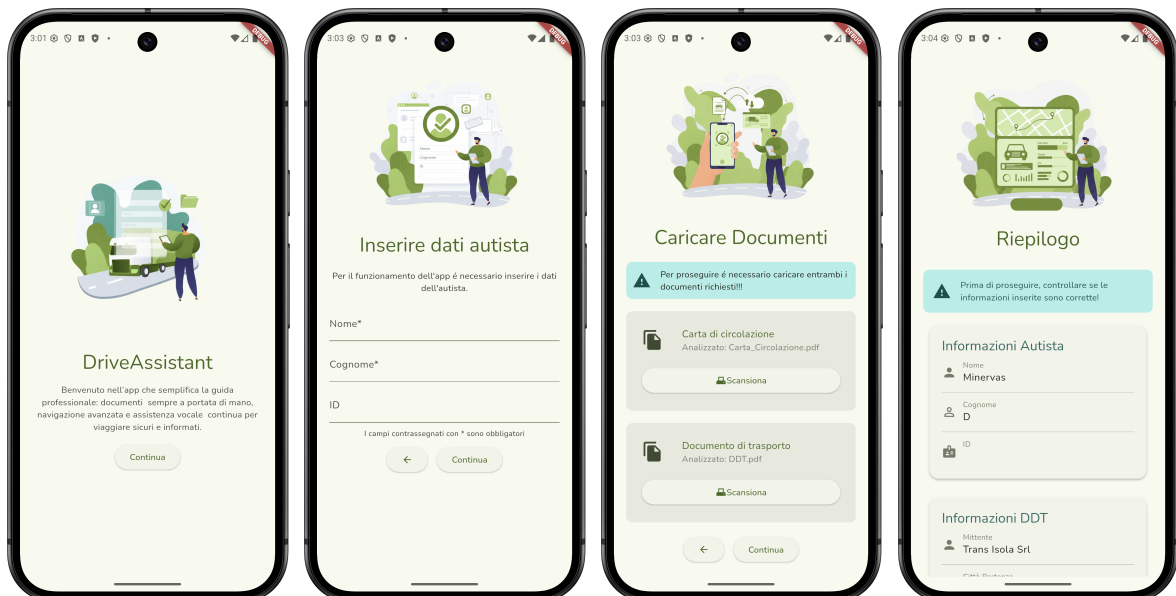


Figura 4.1: Schermate del flusso di *Onboarding* implementato: introduzione, inserimento dei dati dell'autista, caricamento dei documenti e riepilogo finale delle informazioni.

4.1 Scenario applicativo

Lo scenario applicativo di riferimento considera un autista di mezzi pesanti che utilizza l'applicazione durante l'intero *ciclo di una consegna*, dalla fase di onboarding fino alla conclusione del viaggio. All'apertura dell'app, se non è presente una sessione attiva, viene avviata la procedura guidata di onboarding (IF-1), mostrata dalla figura 4.1, che richiede l'inserimento dei dati personali e la scansione dei documenti necessari, supportando costantemente l'utente con istruzioni vocali.

Completato l'onboarding, il sistema avvia automaticamente la navigazione verso la destinazione estratta dal *documento di trasporto* caricato: effettua il geocoding dell'indirizzo e calcola il percorso ottimale verso la meta. Il percorso viene mostrato sulla mappa integrata e il *Navigation SDK* avvia le indicazioni turn-by-turn, affiancate da feedback vocali generati dal servizio TTS per ridurre al minimo la distrazione visiva. L'autista ha ora accesso alle schermate principali (mostrate in figura 4.2), da cui può consultare i documenti caricati, lo stato della consegna e le informazioni sul viaggio, in accordo con i casi d'uso UC-2.2 e UC-2.3.

Durante la guida, l'applicazione mantiene uno stato di supporto vocale continuo: l'autista può ricevere avvisi su cambi di percorso, condizioni del traffico o pericoli e interagire tramite comandi vocali, sfruttando i moduli STT presenti sul backend (in linea con i requisiti IF-3 e IF-6.3).

In caso di errore nel geocoding o di documento non valido, vengono attivate le estensioni dei casi d'uso che mostrano *messaggi di errore* dedicati e permettono l'inserimento manuale dell'indirizzo o la ripetizione della scansione, preservando la continuità del flusso operativo. Al termine della consegna, l'autista seleziona la funzione di conclusione della sessione, il sistema cancella la sessione corrente e riporta l'utente alla schermata iniziale, predisponendo l'app per un *nuovo viaggio* nel rispetto dei requisiti IF-7 e IF-8.

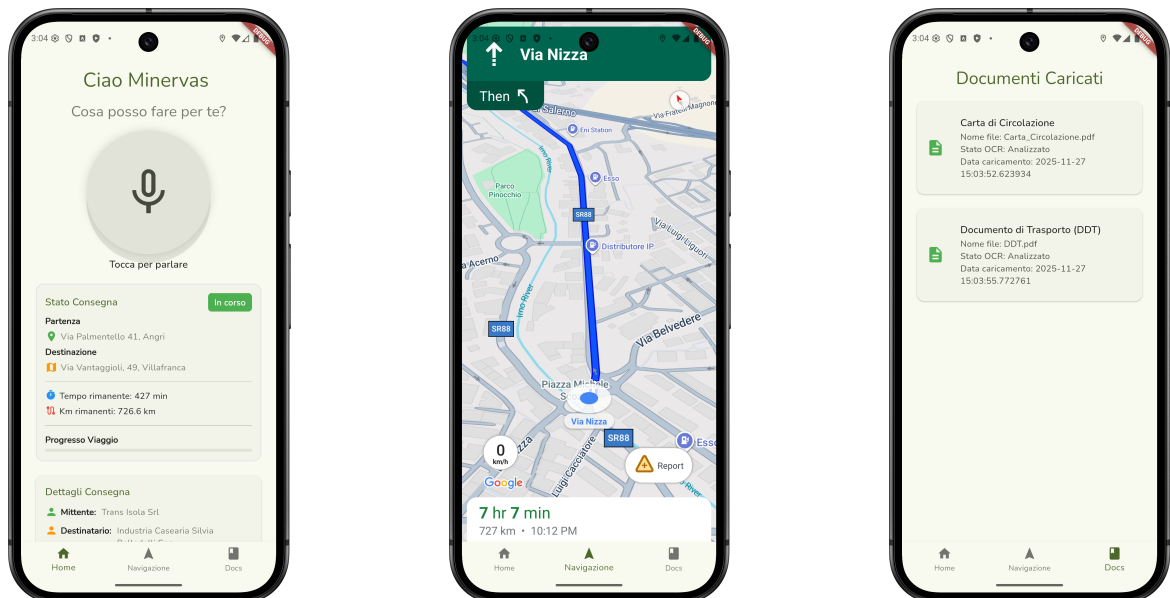


Figura 4.2: Schermate principali dell'applicazione: homepage, interfaccia di navigazione assistita e sezione dedicata alla gestione dei documenti.

4.2 Dati

Il prototipo fa uso di tre principali categorie di dati:

- i dati anagrafici dell'autista
- i documenti necessari alla fase di onboarding
- i dati operativi di viaggio e consegna

I dati anagrafici comprendono informazioni di identificazione di base, utilizzate per associare ogni sessione di guida a un conducente specifico e per popolare le schermate riepilogative dell'applicazione.

Il sistema gestisce 2 tipologie di documenti:

- il documento di trasporto (DDT)
- la carta di circolazione

Entrambi vengono acquisiti tramite scansione nativa o caricamento di file pre-scansionati, in linea con i requisiti [IF-1.1](#) e [IF-1.2](#). Le immagini vengono inviate ai servizi OCR di backend, che estraggono le informazioni strutturate (es. indirizzo di destinazione, targa, dati del veicolo) e le restituiscono al frontend in formato JSON (un esempio di risposta è riportata nella figura [4.3](#)) per la successiva elaborazione. La figura [4.3](#) mostra la struttura di una risposta in formato JSON.

```
1 {  
2   "status": "success",  
3   "pages": [  
4     {  
5       "page": 1,  
6       "result": {  
7         "fundamentals": {  
8           "destinatario": "",  
9           "mittente": "",  
10          "indirizzo_destinatario": "",  
11          "indirizzo_mittente": "",  
12          "citta_destinatario": "",  
13          "citta_mittente": "",  
14          "massa_totale_carico": "NOT FOUND"  
15        },  
16        "extras": {}  
17      }  
18    ]  
19  ]  
20 }
```

Figura 4.3: Esempio della struttura di una risposta (censurata) JSON restituita dal servizio OCR per un singolo documento

I dati di viaggio includono posizione GPS corrente, percorso di navigazione, stato della consegna e storico delle tappe raggiunte. Tali informazioni vengono aggiornate dinamicamente tramite i servizi di geolocalizzazione del dispositivo e le Google Maps APIs, che forniscono

geocoding, calcolo itinerario e navigazione turn-by-turn, in accordo con i requisiti IF-4, IF-5 e IF-6. Lo stato della sessione di guida (attiva, conclusa, sospesa) è mantenuto nel modello applicativo e può essere salvato o resettato secondo i casi d'uso UC-2.1–UC-2.4.

4.3 Verifica e validazione dei requisiti

Al termine della fase di sviluppo, è stata effettuata una *verifica di conformità* per accertare la *copertura funzionale* del sistema. La Tabella 4.1 (*Matrice di Tracciabilità*) mette in relazione i requisiti funzionali, definiti durante l'analisi, con i rispettivi casi d'uso implementati nel prototipo. Tale mappatura dimostra che ogni specifica richiesta trova riscontro in un *flusso operativo concreto* gestito dall'applicazione.

Tabella 4.1: Matrice di Tracciabilità: Requisiti–Casi d'Uso

ID Req.	Descrizione Requisito	Caso d'Uso Correlato
IF-1	Procedura guidata di onboarding	UC-1
IF-1.1	Scansione nativa documenti (DDT, carta di circolazione)	UC-1.2
IF-1.2	Caricamento documenti pre-scansionati	UC-1.2
IF-1.3	Cancellazione/Sostituzione di un documento caricato	UC-1.2, UC-2.3
IF-1.4	Identificazione dell'autista mediante dati personali	UC-1.1
IF-2	Visualizzazione documenti caricati	UC-2.3
IF-3	Supporto vocale continuo	UC-2.1, UC-2.2
IF-3.1	Parola di attivazione per l'assistente vocale	UC-2.1, UC-2.2
IF-4	Visualizzazione stato consegna	UC-2.2
IF-5	Visualizzazione informazioni viaggio	UC-1.3, UC-2.2
IF-6	Sistema completo di navigazione	UC-2.1
IF-6.1	Geocoding indirizzo	UC-2.1
IF-6.2	Navigazione turn-by-turn	UC-2.1
IF-6.3	Notifiche di navigazione e pericoli	UC-2.1, UC-2.2
IF-6.4	Aggiornamento dinamico del percorso	UC-2.1
IF-7	Salvataggio temporaneo della sessione di guida corrente	UC-2.1, UC-2.2
IF-8	Cancellazione/Conclusione della sessione di guida corrente	UC-2.4
UI-1	GUI semplice e minimale	UC-1, UC-2

Continua nella pagina successiva...

ID Req.	Descrizione Requisito	Caso d'Uso Correlato
UI-2	Layout responsive/adaptive	UC-1, UC-2
UI-3	Modalità notturna	UC-2.1, UC-2.2
DF-1	Interazione con il backend tramite dati in formato JSON	UC-1, UC-2
IN-1	Integrazione con servizi di backend containerizzati esterni	UC-1, UC-2

4.4 Conclusioni e sviluppi futuri

In prospettiva, il prototipo può evolvere in una soluzione più completa introducendo, ad esempio la *persistenza storica delle sessioni di guida* e l'analisi dei dati raccolti per finalità di monitoraggio, reportistica e ottimizzazione logistica.

Ciò consisterebbe nella progettazione di un sistema dati più ampio e strutturato, basato su un *database* dedicato alla gestione degli autisti e della flotta di mezzi. L'introduzione di un *modello relazionale* ¹ consentirebbe di definire legami più espressivi tra le diverse entità, come l'associazione di uno storico dettagliato delle tratte effettuate, degli eventi di navigazione e degli esiti delle consegne a ciascun conducente o veicolo, abilitando così funzionalità avanzate di tracciamento e analisi delle performance.

Un'ulteriore area di miglioramento riguarda la gestione della connettività: nella versione attuale, l'applicazione richiede una connessione Internet sempre attiva, in quanto dipende dai servizi cloud di navigazione e geocoding. In scenari futuri si potrebbe prevedere un meccanismo di *fallback* in modalità degradata, capace di garantire almeno le funzionalità essenziali anche in assenza di rete, riducendo l'impatto delle interruzioni di servizio sull'operatività dell'autista.

Per quanto riguarda invece l'esperienza d'uso e la personalizzazione del sistema, potrebbe essere introdotto il *supporto multilingua* per adattare interfaccia, messaggi vocali e testi ai diversi contesti geografici in cui operano gli autisti. Parallelamente, si potrebbe progettare un sistema completo di *registrazione e autenticazione* dell'utente, basato su credenziali personali e, dove disponibile, su meccanismi biometrici (impronta digitale o riconoscimento facciale) per semplificare e velocizzare l'accesso dei conducenti.

¹*Modello relazionale*: modello logico di organizzazione dei dati proposto da *E.F. Codd* nel 1970, basato sulla teoria matematica degli insiemi. In questo modello, i dati sono strutturati in relazioni (comunemente rappresentate come tabelle), composte da *tuple* (righe) e attributi (colonne).

Bibliografia

- [1] Jeanne Breen Consulting. *Car telephone use and road safety. Final Report. An overview prepared for the European Commission*. Prepared by Jeanne Breen Consulting. European Commission, giu. 2009.
- [2] European Road Safety Observatory. *Road Safety Thematic Report – Driver distraction*. European Commission, 2022.
- [8] V. Giacchina. *Sviluppare applicazioni con Flutter: Guida al framework e al linguaggio Dart per lo sviluppo mobile cross platform*. Feltrinelli Editore, 2023.
- [17] PlantUML Team. *PlantUML Language Reference Guide*. Versione 1.2025.0. PlantUML. 2025. URL: <https://plantuml.com/guide>.
- [19] Frank Tsui, Orlando Karam e Barbara Bernal. *Essentials of software engineering*. Jones & Bartlett Learning, 2022.
- [20] Martin Fowler. *UML Distilled: Guida rapida al linguaggio di modellazione standard*. Pearson Italia Spa, 2004.

- [3] Figma. *What is Figma?* URL: <https://help.figma.com/hc/en-us/articles/14563969806359-What-is-Figma>.
- [4] Miro. *Academy*. URL: <https://academy.miro.com/>.
- [5] Dart. *Overview*. URL: <https://dart.dev/>.
- [6] Flutter. *Docs*. URL: <https://flutter.dev/>.
- [7] RadixWeb. URL: <https://radixweb.com/blog/overview-of-flutter-architecture>.
- [9] Kotlin. *Docs*. URL: <https://kotlinlang.org/docs/home.html>.
- [10] Flutter. *Documentation*. URL: <https://www.swift.org/documentation/>.
- [11] Android Studio. URL: <https://developer.android.com/studio>.
- [12] Xcode. *Docs*. URL: <https://developer.apple.com/xcode/>.
- [13] Gradle. *Getting Started*. URL: https://docs.gradle.org/current/userguide/getting_started_eng.html.
- [14] Gradle. *Gradle Kotlin DSL Primer*. URL: https://docs.gradle.org/current/userguide/kotlin_dsl.html.
- [15] Git. *What is Git?* URL: <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>.
- [16] Github. URL: <https://github.com/>.
- [18] Google Maps APIs. URL: <https://developers.google.com/maps/apis-by-platform?hl=it>.
- [21] ABP. URL: <https://abp.io/community/articles/color-psychology-in-web-design-how-to-choose-the-perfect-palette-in-2025-z383jph8#gsc.tab=0>.
- [22] MinervaS. *Chi Siamo*. URL: <https://www.minervas.it/chi-siamo/>.